

Simulating Spectral Sampling of the ExoMars Mission Reference Samples with Enfys

October 3, 2025

1 Abstract

This notebook logs the resampling of high-resolution spectrometer measurements of the ExoMars Mission Reference Samples for the purposes of simulating measurements with the Enfys flight model.

In this notebook we assume Enfys uses a modified InGaAs Mid-Wave Infrared (1.6 - 2.5 μm , ‘MWIR’) sensor, that is subject to thermal noise from the Enfys structure.

2 Introduction

As discussed in [previous notebooks](#), Enfys is an Infra-Red point-spectrometer for the ExoMars Rosalind Franklin rover.

In this notebook the objective is to produce 2 datasets of Enfys-like data, one without additive noise, and one with.

Analysis of the data is not within the scope of this notebook.

We simulate the expected performance of Enfys using modelled performance data for an InGaAs Short-Wave infrared sensor (0.9 - 1.7 μm) and a modified InGaAs Medium-Wave Infrared sensor for the wavelength range of 1.6 - 2.5 μm . Included in the modelling is the noise expected for the sensor, as provided by the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 document (contact Enfys P.I. M. Gunn).

2.1 Problem

[TODO overview of the ExoMars Mission Reference Samples]

3 Method

[TODO description of how the data was collected and by what instruments, and how that data is aggregated and resampled here using SPTK.]

3.1 Spectral Resampling

We take a simple approach to resampling, as follows.

We neglect reflectance calibration uncertainty, and any systematic uncertainty when merging the data from the two separate photodiodes, and assume that the linear-variable filter (LVF) movement is perfectly calibrated such that a measurement at centre-wavelength λ_{cwl} can be requested with arbitrary precision.

We assume an observation of λ_{cwl} has a Cauchy transmission profile. We use the latest expectations of the spectral resolving power of the short-wave infrared (SWIR) and mid-wave infrared (MWIR) LVFs, that we have linearly interpolated from ~ 30 measurements from 0.9 - 2.6 μm to arbitrary values of λ in the SWIR and MWIR ranges, to give the vector $\Gamma[\lambda]$. The Cauchy transmission profile for a given λ_{cwl} has a full-width-at-half-maximum of $\Delta\lambda = \lambda_{cwl}/\Gamma[\lambda_{cwl}]$.

This data has been digitised from the plot reported in the `Enfys schedule review 20250807.ppt`, by Matt Gunn.

```
[188]: import pandas as pd

enfys_swir_respow = pd.read_csv('../data/instruments/enfys_swir_respow.csv',
    ↪index_col=0)
enfys_mwir_respow = pd.read_csv('../data/instruments/enfys_mwir_respow.csv',
    ↪index_col=0)
g=enfys_swir_respow.plot(label='SWIR', marker='o')
enfys_mwir_respow.plot(marker='o',label='MWIR', xlabel='Wavelength (nm)',
    ↪ylabel=r'\lambda/\Delta\lambda$ Resolving Power',ax=g)
g.legend(['SWIR', 'MWIR'])
title = g.set_title('Enfys SWIR & MWIR Spectral Resolving Power
    ↪\Gamma[\lambda]$')
```

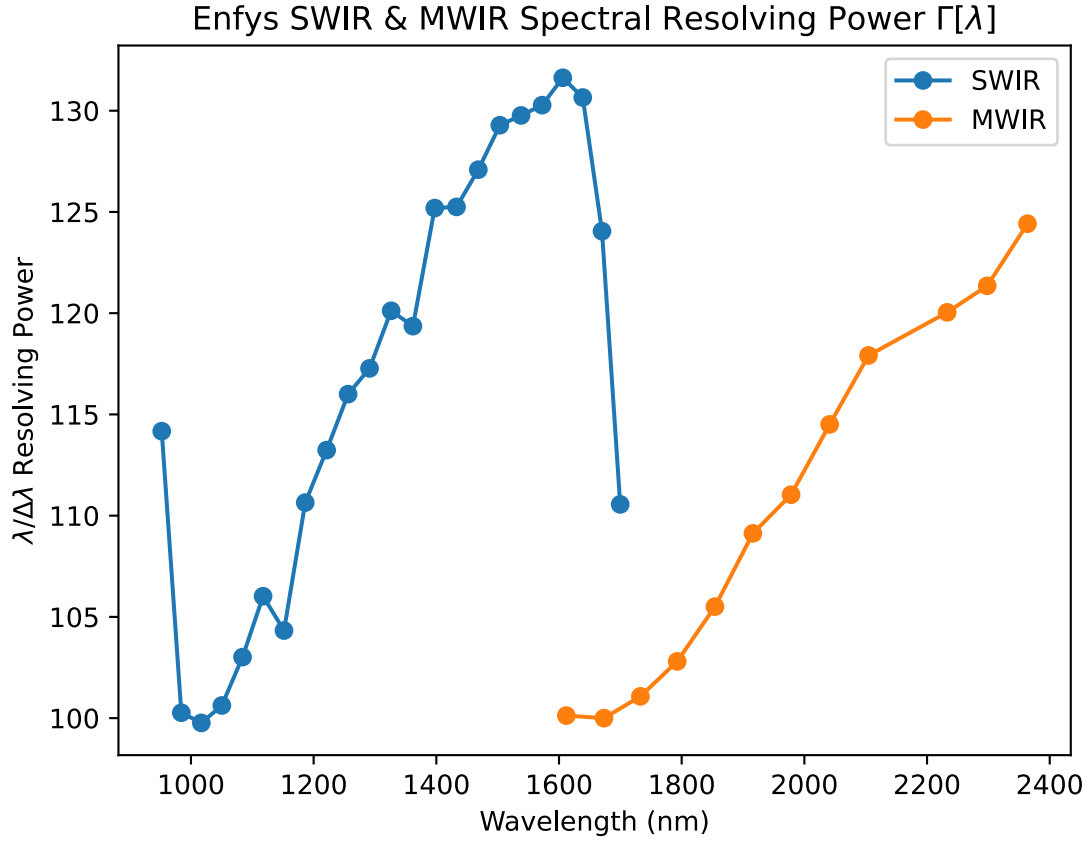


Figure 1 Enfys SWIR & MWIR Spectral Resolving Power $\Gamma[\lambda]$, extracted from `Enfys_schedule_review_20250807.ppt`, M. Gunn.

The Cauchy transmission profile $T(\lambda|\lambda_{cwl})$ is given by

$$T(\lambda|\lambda_{cwl}) = \frac{\gamma^2}{(\lambda - \lambda_{cwl}^2) + \gamma^2}$$

where $\gamma = \Delta\lambda/2 = (\lambda_{cwl}/\Gamma[\lambda_{cwl}])/2$.

The shape of the Cauchy profile is illustrated here.

```
[189]: from sptk.instrument import Instrument
import sptk.config as cfg
from matplotlib import pyplot as plt

cfg.update_sample_res('wvl_min', 800)
cfg.update_sample_res('wvl_max', 2600)

exmpl_cauchy = Instrument.build_cauchy_filter(1600, 1600/100)
plt.plot(cfg.WVLS, exmpl_cauchy.squeeze())
```

```
plt.xlim(1500, 1700)
plt.xlabel('Wavelength (nm)')
plt.ylabel('Relative Transmittance')
title = plt.title('Example Cauchy Filter at 1600 nm with Spectral Resolving Power  $\Gamma = 100$ ')
plt.show()
```

Example Cauchy Filter at 1600 nm with Spectral Resolving Power $\Gamma = 100$

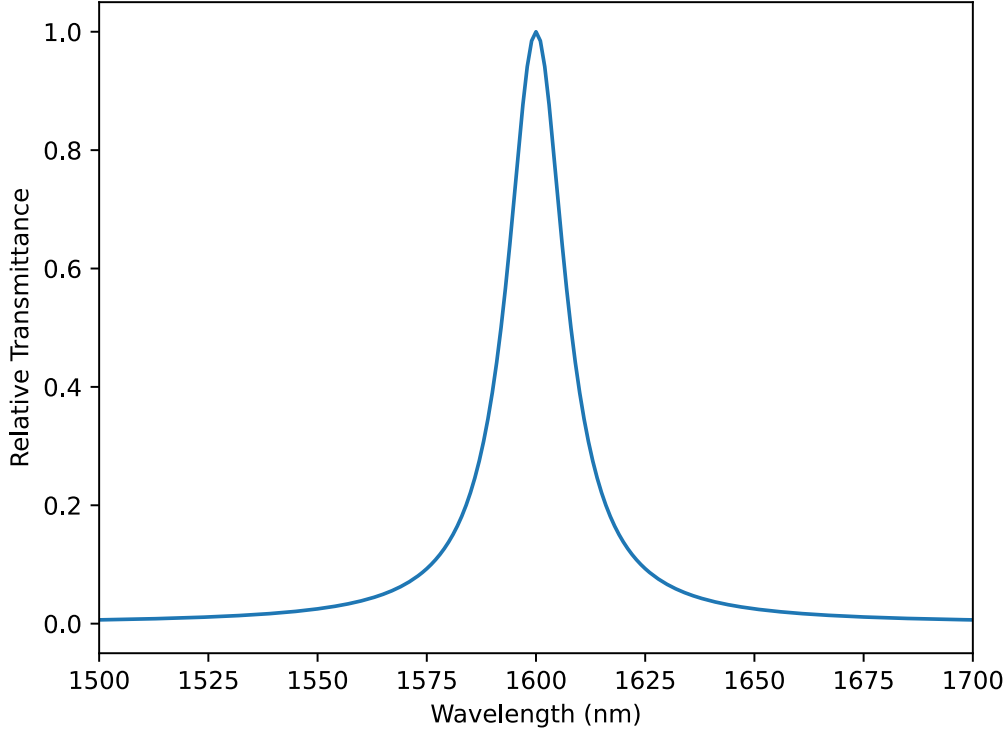


Figure 2 Example Cauchy Filter transmission profile, at 1600 nm with Spectral Resolving Power $\Gamma = 100$.

We assume that a high-resolution laboratory spectrum approximates the continuous function $R(\lambda)$, such that the calibrated reflectance at λ_{cwl} has the expected value $R[\lambda_{cwl}]$ of:

$$R[\lambda_{cwl}] = \frac{\int R(\lambda)T(\lambda|\lambda_{cwl})d\lambda}{\int T(\lambda|\lambda_{cwl})d\lambda}$$

We use this equation to compute the Enfys-sampled reflectance spectra.

3.2 Adding Noise

To simulate noise we add ε to a reflectance measurement, an additive noise term drawn from the Gaussian distribution with standard deviation σ , that we define relative to the specified Signal-to-Noise Ratio (SNR),

$$\sigma = \frac{S}{\text{SNR}}$$

where S is the signal.

The supplied Signal-to-Noise Ratio vector has been defined under the assumption of a 30% albedo spectrally uniform reflector.

The expected spectral SNR of the SWIR and MWIR sensors has been measured and processed by P.I. M. Gunn, and reported in the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 spreadsheet.

We read in the SNR file, and get the SNR data for the InAs MWIR sensor and the InGaAs SWIR sensor,

```
[190]: enfys_snr = pd.read_csv('../data/instruments/enfys_snr.csv', index_col=0)
        ingaas_mwir_snr = enfys_snr['InGaAs MWIR SNR']
        ingaas_swir_snr = enfys_snr['InGaAs SWIR SNR']

        g = ingaas_swir_snr.plot(grid=True, label='Nominal InGaAs SWIR SNR')
        ingaas_mwir_snr.plot(logy=True, grid=True, label='Nominal InGaAs MWIR SNR')
        g.set_xlabel('Wavelength (nm)')
        g.set_ylabel('SNR')
        g.legend()
        title = g.set_title('Spectral SNR of InGaAs SWIR and InGaAs MWIR Sensors')
```

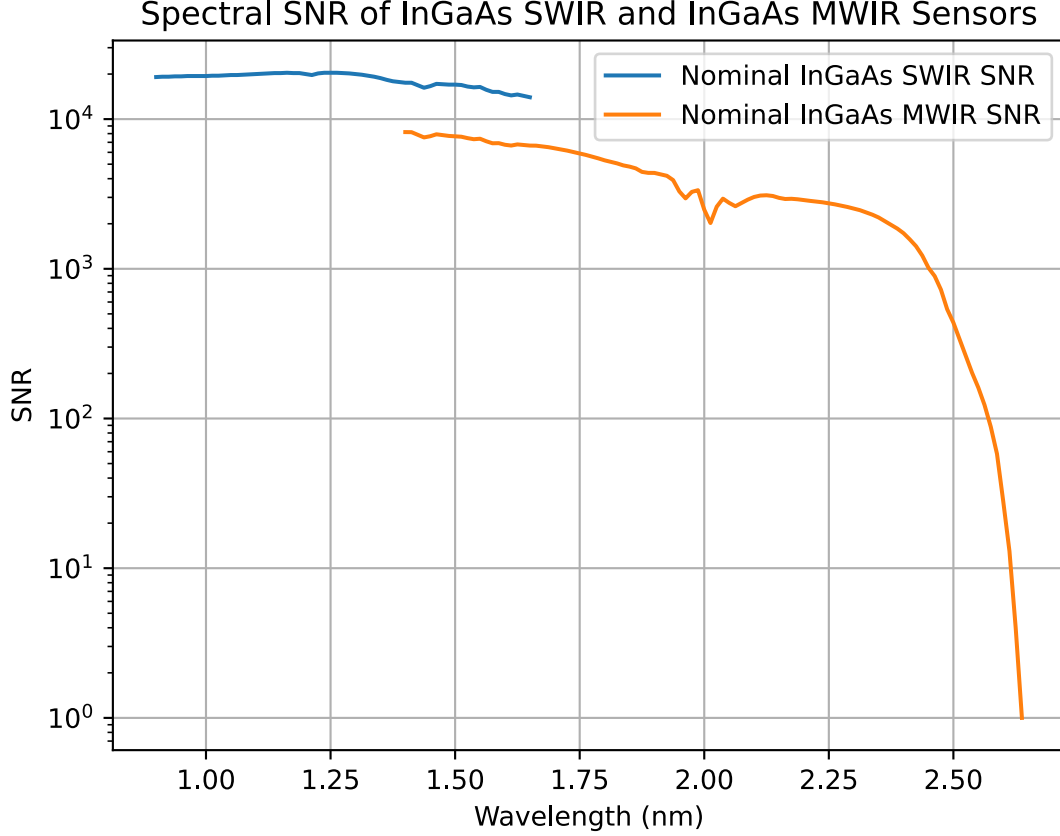


Figure 3 Spectral Signal-to-Noise Ratio for InGaAs SWIR and InGaAs MWIR Sensors, extracted from EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 spreadsheet, M. Gunn.

We assume that this SNR vector is modulated by the reflectance of the mineral of interest, such that σ as a function of the spectral Signal-to-Noise Ratio $\text{SNR}[\lambda]$ is:

$$\sigma[\lambda] = \frac{R[\lambda]}{\text{SNR}[\lambda]}$$

For each observation $R[\lambda_{cwl}]$ we draw n -repeat noisy samples $\tilde{R}_i[\lambda_{cwl}]$, such that

$$\tilde{R}_i[\lambda_{cwl}] = R[\lambda_{cwl}] + \varepsilon_i[\lambda_{cwl}]$$

with

$$\varepsilon_i[\lambda_{cwl}] \sim \mathcal{N}(0, \sigma[\lambda_{cwl}])$$

This gives an indication of the expected noise of a calibrated *Enfys* acquired spectrum.

3.3 Analysis

First we produce plots of the resampled spectra, and single instances of noisy spectra, and visually inspect these to consider the ability of this EnfyS configuration to resolve diagnostic spectral features against the noise.

3.3.1 Spectral Angle

We then take a single noisy sample and evaluate the Spectral Angle between the noisy sample and each noise-free entry of the MICA files spectral database that we are sampling. We repeat this for each noisy sample, to give the mean Spectral Angle and it's uncertainty, to determine the ability to discriminate between similar mineral reflectance spectra.

The Spectral Angle is defined¹ between the noisy spectrum $\tilde{R}_i[\lambda]$ of material i in the set of MICA file materials ($\mathcal{M}$) and the clean spectrum $R_j[\lambda]$, where $j \in \mathcal{M}$.

The Spectral Angle θ is:

$$\theta(i, j) = \arccos \left(\frac{\sum_{\lambda} \tilde{R}_i[\lambda] R_j[\lambda]}{(\sum_{\lambda} \tilde{R}_i[\lambda]^2)^{1/2} (\sum_{\lambda} R_j[\lambda]^2)^{1/2}} \right)$$

This closed form expression can be formulated in a way such that θ can be computed for all $\mathcal{M}$ for a given material i in parallel, as well as multiple noisy repeat samples of i .

3.4 Notebook and Environment Setup

Here we include the required code blocks for setting up this notebook for this study.

```
[191]: %load_ext autoreload
%autoreload 2
%config InlineBackend.figure_format = 'svg' # note - requires Inkscape to be
      ↪ installed, and to be accessible from the terminal
# on macos, add inkscape to the path with `sudo ln -s /Applications/Inkscape.
      ↪ app/Contents/MacOS/inkscape /usr/local/bin/inkscape`
%matplotlib inline
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

We will be using the Spectral Parameters Toolkit. First we import the required modules.

```
[192]: import sptk.config as cfg
from sptk.material_collection import MaterialCollection
from sptk.instrument import Instrument
from sptk.observation import Observation
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla
```

¹F.A. Kruse, A.B. Lefkoff, J.W. Boardman, K.B. Heidebrecht, A.T. Shapiro, P.J. Barloon, A.F.H. Goetz, The spectral image processing system (SIPS)—interactive visualization and analysis of imaging spectrometer data, Remote Sensing of Environment, Volume 44, Issues 2–3, 1993, Pages 145–163, ISSN 0034-4257, [https://doi.org/10.1016/0034-4257\(93\)90013-N](https://doi.org/10.1016/0034-4257(93)90013-N)

We set our simulation resolution range to extend the range of Enfys, of 0.9 - 2.5 μm , such that the long tails of the Cauchy distribution of the linear-variable-filter profiles can be accommodated during sampling. We leave our spectral resolution at 1 nm for all λ .

```
[193]: cfg.update_sample_res('wvl_min', 800)
       cfg.update_sample_res('wvl_max', 2600)
```

Finally we set the project name.

```
[194]: PROJECT_NAME = 'enfys_mission_reference_samples'
```

4 Data

TODO - better write up of this.

Data was collected at Aber. Uni. by Enfys team (PG, CC, LP).

Here we use data from the visible-to near infrared spectrometer.

First we prepare an SPTK compatible spectral library from the data, and then we ingest this library into the notebook via the Material Collection class.

4.1 Preparing the Dataset

The data is held adjacent to this directory

```
[195]: from pathlib import Path
       raw_data_dir = '../..../enfys_data/RS-3500_spectrometer/'
       raw_data_files = sorted(list(Path(raw_data_dir).glob('*.*sed')))
```

Let's try to read one of these files into the notebook, using pandas.

```
[196]: import pandas as pd
       from sptk.material_collection import header_template
       from sptk.material_collection import MaterialCollectionBuilder
```

```
[197]: library = 'EXM_MRS'
```

```
[198]: def create_material_entry(raw_data_file: Path, spectral_library: str) ->
       ↪tuple[dict, pd.Series]:
       """
       Create a material entry from a raw .sed file from the ExoMars Mission
       ↪Reference Samples dataset.

       :param raw_data_file: path to the raw .sed file
       :type raw_data_file: Path
       :return: header dictionary and series of reflectance data
       :rtype: tuple(dict, pd.Series)
       """
       # read the raw text file
```



```

    # it's not so efficient to read each file twice, but we don't have 1000's
    ↪to read, so it's ok
    raw_hdr = pd.read_csv(raw_data_file, header=None, nrows=26, sep=': ',
    ↪index_col=0, engine='python')
    raw_data = pd.read_csv(raw_data_file, header=26, sep='\t', index_col=0,
    ↪engine='python')

    # extract the reflectance data
    raw_refl = raw_data['Reflect. %']

    # write this information to a Material Collection compatible .csv file.
    # this should really be a class/function of the MaterialCollection module,
    ↪in the same way we have an InstrumentBuilder class of the Instrument module.
    header = header_template()

    # gathering header metadata
    sample_id = '_'.join(raw_data_file.stem.split('_')[1:])
    species = '_'.join(raw_data_file.stem.split('_')[1:-1])
    # we will use the species name as the group and subgroup
    group = species
    subgroup = species
    sample_description = raw_hdr.loc['Comment'].values[0]
    date_added = raw_hdr.loc['Date'].values[0]
    database_of_origin = 'ExoMars Mission Reference Samples'
    other_information = f"Acquired with {raw_hdr.loc['Instrument']}; {raw_hdr.
    ↪loc['Integration'].values[0]} integration times; {raw_hdr.loc['Averages'].
    ↪values[0]} scans averaged"
    resolution = "2.8nm@700nm 8nm@1500nm 6nm@2100nm"
    # putting the information in the header dictionary
    header['Sample ID'] = sample_id
    header['Species'] = species
    header['Subgroup'] = subgroup
    header['Group'] = group
    header['Library'] = library
    header['Sample Description'] = sample_description
    header['Date Added'] = date_added
    header['Database of Origin'] = database_of_origin
    header['Resolution'] = resolution
    header['Other Information'] = other_information

    raw_refl.index.name = 'Wavelength'
    raw_refl.name = 'Response'
    raw_refl = raw_refl / 100 # convert from % to a fraction

    return header, raw_refl

```

```
[199]: exm_mrs_lib = MaterialCollectionBuilder(library)
```

```
[201]: for raw_data_file in raw_data_files:
        header, data = create_material_entry(raw_data_file, library)
        sample_id = header['Sample ID']
        print(f"Adding {sample_id} to {library} library")
        exm_mrs_lib.add_material(header, data)
```

```
Adding 10_Otago_00001 to EXM_MRS library
Adding 10_Otago_00002 to EXM_MRS library
Adding 10_Otago_00003 to EXM_MRS library
Adding 10_Otago_00004 to EXM_MRS library
Adding 11_Iceland_00001 to EXM_MRS library
Adding 11_Iceland_00002 to EXM_MRS library
Adding 11_Iceland_00003 to EXM_MRS library
Adding 11_Iceland_00004 to EXM_MRS library
Adding 15_Oxia_Sophia_00001 to EXM_MRS library
Adding 15_Oxia_Sophia_00002 to EXM_MRS library
Adding 15_Oxia_Sophia_00003 to EXM_MRS library
Adding 15_Oxia_Sophia_00004 to EXM_MRS library
Adding 2_Atacama_Red_00001 to EXM_MRS library
Adding 2_Atacama_Red_00002 to EXM_MRS library
Adding 2_Atacama_Red_00003 to EXM_MRS library
Adding 2_Atacama_Red_00004 to EXM_MRS library
Adding 3_Sabtha_00001 to EXM_MRS library
Adding 3_Sabtha_00002 to EXM_MRS library
Adding 3_Sabtha_00003 to EXM_MRS library
Adding 3_Sabtha_00004 to EXM_MRS library
Adding 4_Watchet_Bay_00001 to EXM_MRS library
Adding 4_Watchet_Bay_00002 to EXM_MRS library
Adding 4_Watchet_Bay_00003 to EXM_MRS library
Adding 4_Watchet_Bay_00004 to EXM_MRS library
Adding 4_Watchet_Bay_00005 to EXM_MRS library
Adding 4_Watchet_Bay_00006 to EXM_MRS library
Adding 5_Granby_00001 to EXM_MRS library
Adding 5_Granby_00002 to EXM_MRS library
Adding 5_Granby_00003 to EXM_MRS library
Adding 5_Granby_00004 to EXM_MRS library
Adding 6_Hydrothermal_Clay_00001 to EXM_MRS library
Adding 6_Hydrothermal_Clay_00002 to EXM_MRS library
Adding 6_Hydrothermal_Clay_00003 to EXM_MRS library
Adding 6_Hydrothermal_Clay_00004 to EXM_MRS library
Adding 7_Oxia_Teresa_00001 to EXM_MRS library
Adding 7_Oxia_Teresa_00002 to EXM_MRS library
Adding 7_Oxia_Teresa_00003 to EXM_MRS library
Adding 7_Oxia_Teresa_00004 to EXM_MRS library
Adding 8_Mudstone_00001 to EXM_MRS library
Adding 8_Mudstone_00002 to EXM_MRS library
Adding 8_Mudstone_00003 to EXM_MRS library
Adding 8_Mudstone_00004 to EXM_MRS library
```

```

Adding 9_Congo_00001 to EXM_MRS library
Adding 9_Congo_00002 to EXM_MRS library
Adding 9_Congo_00003 to EXM_MRS library
Adding 9_Congo_00004 to EXM_MRS library
Adding Standard_00001 to EXM_MRS library
Adding Standard_00002 to EXM_MRS library
Adding Standard_00003 to EXM_MRS library
Adding Standard_retest_00001 to EXM_MRS library
Adding Standard_retest_00002 to EXM_MRS library
Adding Standard_retest_00003 to EXM_MRS library

```

Now we need to construct a material set for categorising these.

```
[202]: exm_mrs_lib_map = exm_mrs_lib.map_spectral_library()
```

```
[203]: exm_mrs_lib_map.keys()
```

```
[203]: dict_keys(['4_Watchet_Bay', '7_Oxia_Teresa', '3_Sabtha', '10_Otago',
'2_Atacama_Red', '9_Congo', 'Standard', '11_Iceland', '6_Hydrothermal_Clay',
'Standard_retest', '15_Oxia_Sophia', '5_Granby', '8_Mudstone'])
```

```
[204]: EXM_MRS_LIB = {}

samples_list = []
for key in exm_mrs_lib_map.keys():
    samples_list.append(key)
# move the 'Standard' and 'Standard_retest' to a separate 'standards' list
EXM_MRS_LIB['standards'] = [(s, '*') for s in samples_list if 'Standard' in s]
EXM_MRS_LIB['samples'] = [(s, '*') for s in samples_list if 'Standard' not in s]
```

```
[205]: EXM_MRS_LIB
```

```
[205]: {'standards': [('Standard', '*'), ('Standard_retest', '*')],
'samples': [('4_Watchet_Bay', '*'),
('7_Oxia_Teresa', '*'),
('3_Sabtha', '*'),
('10_Otago', '*'),
('2_Atacama_Red', '*'),
('9_Congo', '*'),
('11_Iceland', '*'),
('6_Hydrothermal_Clay', '*'),
('15_Oxia_Sophia', '*'),
('5_Granby', '*'),
('8_Mudstone', '*')]]}
```

4.2 Material Collection Construction

We use the `sptk.material_collection` module to parse the spectral library:

```
[206]: matcol = MaterialCollection(
    EXM_MRS_LIB,
    'EXM_MRS',
    PROJECT_NAME,
    load_existing=False,
    balance_classes=False,
    random_bias_seed=None,
    allow_out_of_bounds=True,
    plot_profiles=False,
    export_df=True)
```

Building new enfys_mission_reference_samples MaterialCollection DF

```
-----
Category
  species|subgroup|group|
    data_id status
-----
Standards
-Standard|Standard|Standard|
  -Standard_00001.csv loaded
  -Standard_00002.csv loaded
  -Standard_00003.csv loaded
-Standard_retest|Standard_retest|Standard_retest|
  -Standard_retest_00001.csv loaded
  -Standard_retest_00002.csv loaded
  -Standard_retest_00003.csv loaded
-----
Samples
-10_Otago|10_Otago|10_Otago|
  -10_Otago_00001.csv loaded
  -10_Otago_00002.csv loaded
  -10_Otago_00003.csv loaded
  -10_Otago_00004.csv loaded
-11_Iceland|11_Iceland|11_Iceland|
  -11_Iceland_00001.csv loaded
  -11_Iceland_00002.csv loaded
  -11_Iceland_00003.csv loaded
  -11_Iceland_00004.csv loaded
-15_Oxia_Sophia|15_Oxia_Sophia|15_Oxia_Sophia|
  -15_Oxia_Sophia_00001.csv loaded
  -15_Oxia_Sophia_00002.csv loaded
  -15_Oxia_Sophia_00003.csv loaded
  -15_Oxia_Sophia_00004.csv loaded
-2_Atacama_Red|2_Atacama_Red|2_Atacama_Red|
  -2_Atacama_Red_00001.csv loaded
  -2_Atacama_Red_00002.csv loaded
  -2_Atacama_Red_00003.csv loaded
```

```

-2_Atacama_Red_00004.csv loaded
-3_Sabtha|3_Sabtha|3_Sabtha|
-3_Sabtha_00001.csv loaded
-3_Sabtha_00002.csv loaded
-3_Sabtha_00003.csv loaded
-3_Sabtha_00004.csv loaded
-4_Watchet_Bay|4_Watchet_Bay|4_Watchet_Bay|
-4_Watchet_Bay_00001.csv loaded
-4_Watchet_Bay_00002.csv loaded
-4_Watchet_Bay_00003.csv loaded
-4_Watchet_Bay_00004.csv loaded
-4_Watchet_Bay_00005.csv loaded
-4_Watchet_Bay_00006.csv loaded
-5_Granby|5_Granby|5_Granby|
-5_Granby_00001.csv loaded
-5_Granby_00002.csv loaded
-5_Granby_00003.csv loaded
-5_Granby_00004.csv loaded
-6_Hydrothermal_Clay|6_Hydrothermal_Clay|6_Hydrothermal_Clay|
-6_Hydrothermal_Clay_00001.csv loaded
-6_Hydrothermal_Clay_00002.csv loaded
-6_Hydrothermal_Clay_00003.csv loaded
-6_Hydrothermal_Clay_00004.csv loaded
-7_Oxia_Teresa|7_Oxia_Teresa|7_Oxia_Teresa|
-7_Oxia_Teresa_00001.csv loaded
-7_Oxia_Teresa_00002.csv loaded
-7_Oxia_Teresa_00003.csv loaded
-7_Oxia_Teresa_00004.csv loaded
-8_Mudstone|8_Mudstone|8_Mudstone|
-8_Mudstone_00001.csv loaded
-8_Mudstone_00002.csv loaded
-8_Mudstone_00003.csv loaded
-8_Mudstone_00004.csv loaded
-9_Congo|9_Congo|9_Congo|
-9_Congo_00001.csv loaded
-9_Congo_00002.csv loaded
-9_Congo_00003.csv loaded
-9_Congo_00004.csv loaded

```

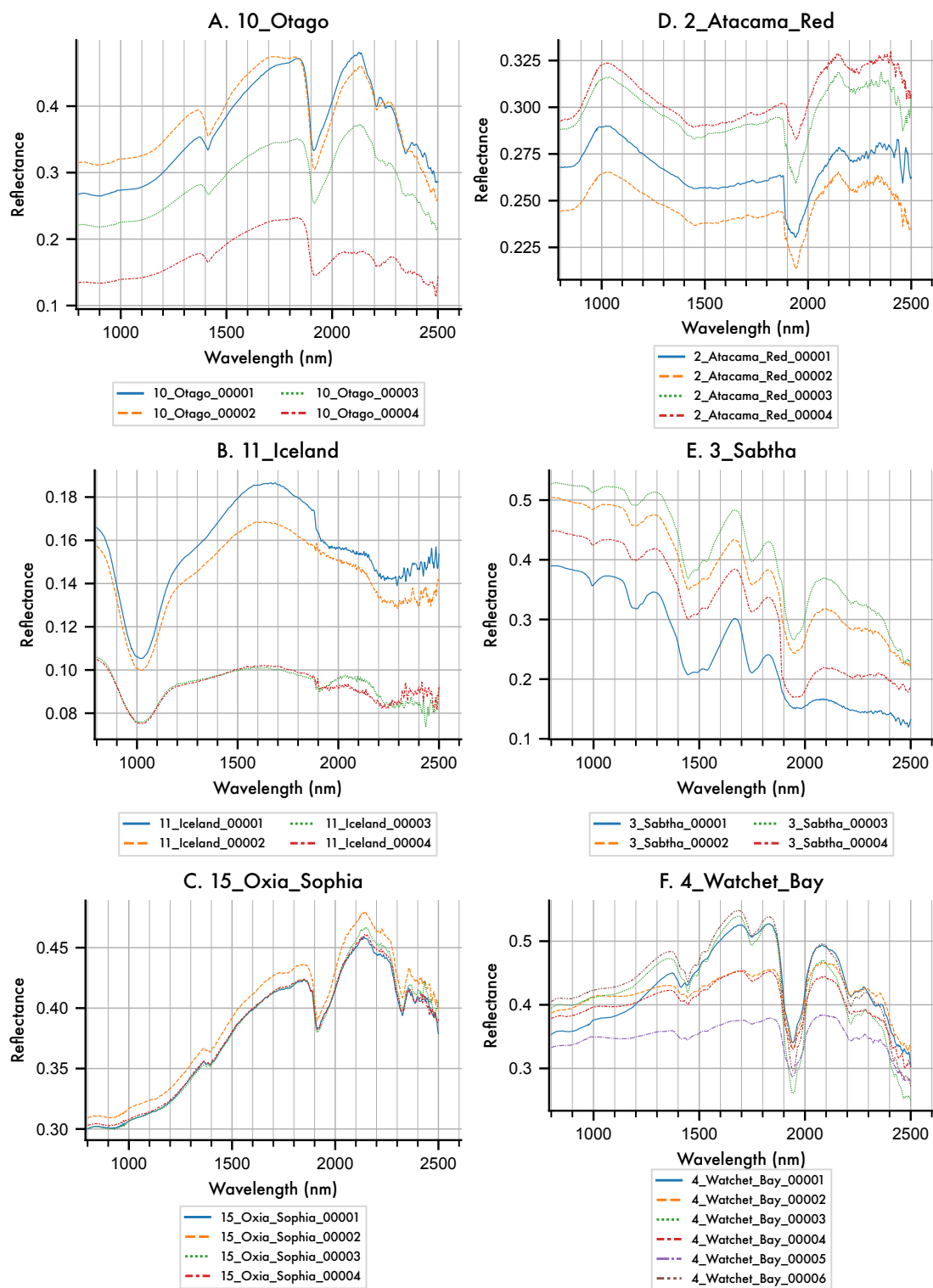
Loading 52 of entries complete.

Exporting the Material Collection to CSV and Pickle formats...

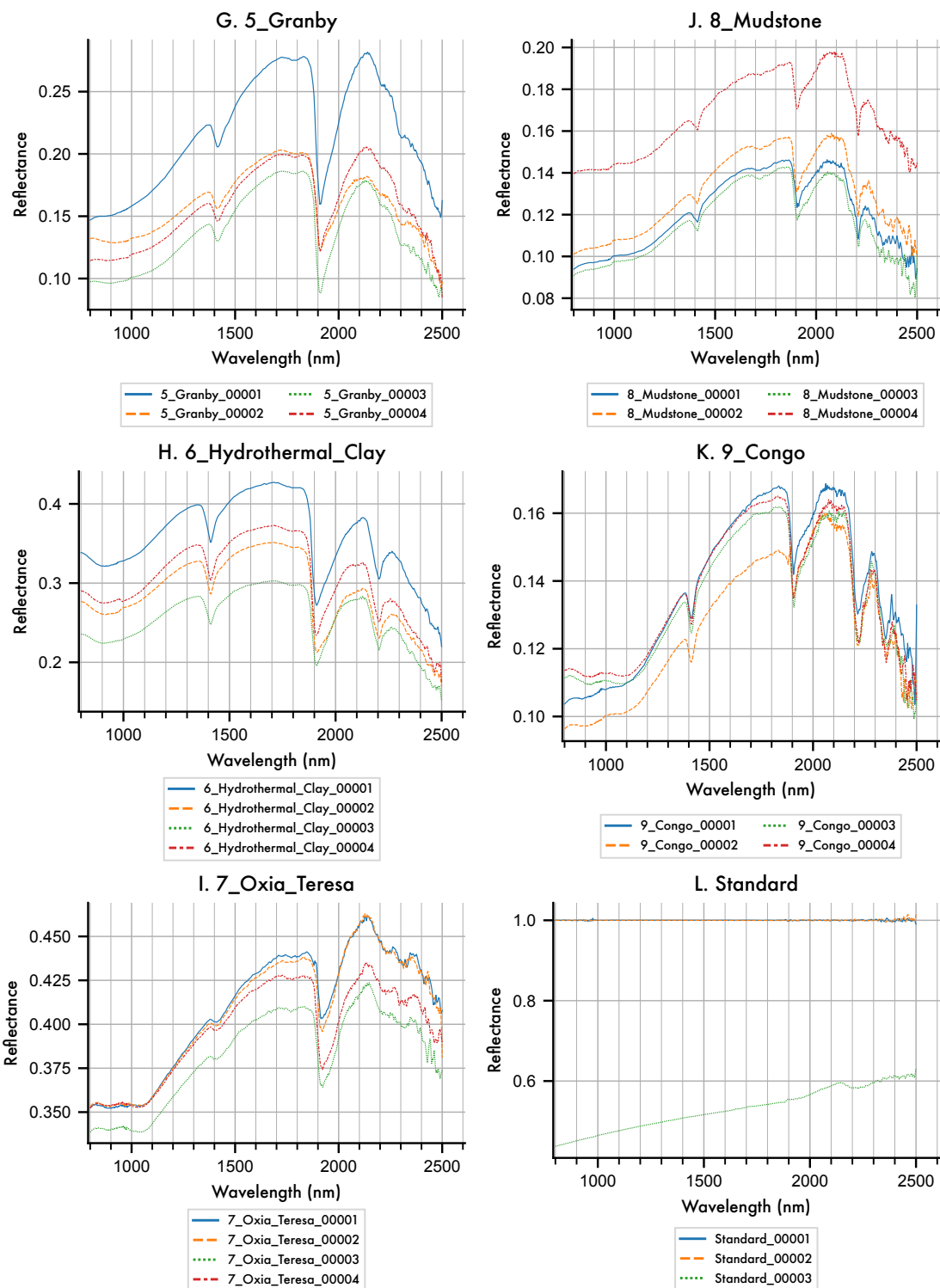
We can use the `material_collection` plotting routines to visualise the library.

```
[226]: ax = matcol.plot_profiles(stacked=False, scope='species', groupby='Sample ID')
```

Species grouped by Sample Id (1/3)



Species grouped by Sample Id (2/3)



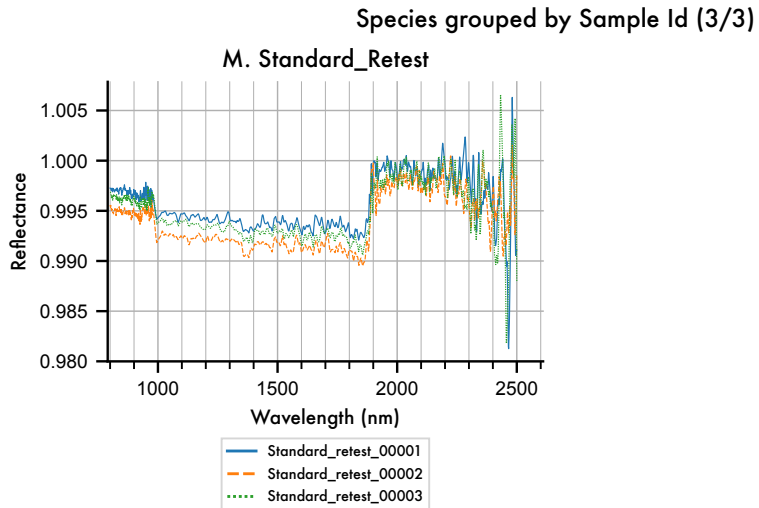


Figure 4 High resolution MICA Files spectral library, arranged by category and grouped by mineral species.

We can see that not all of the entries cover the complete 0.9 - 3.1 μm range of *Enfys*, so in future work we should look to replace these MICA files laboratory type spectra with spectra that all span the entire range.

We can show band locations more clearly after continuum removal. We can use the `SpectralLibraryAnalyser.remove_continuum()` function to produce a duplicate of the `MaterialCollection` but with the data continuum-removed.

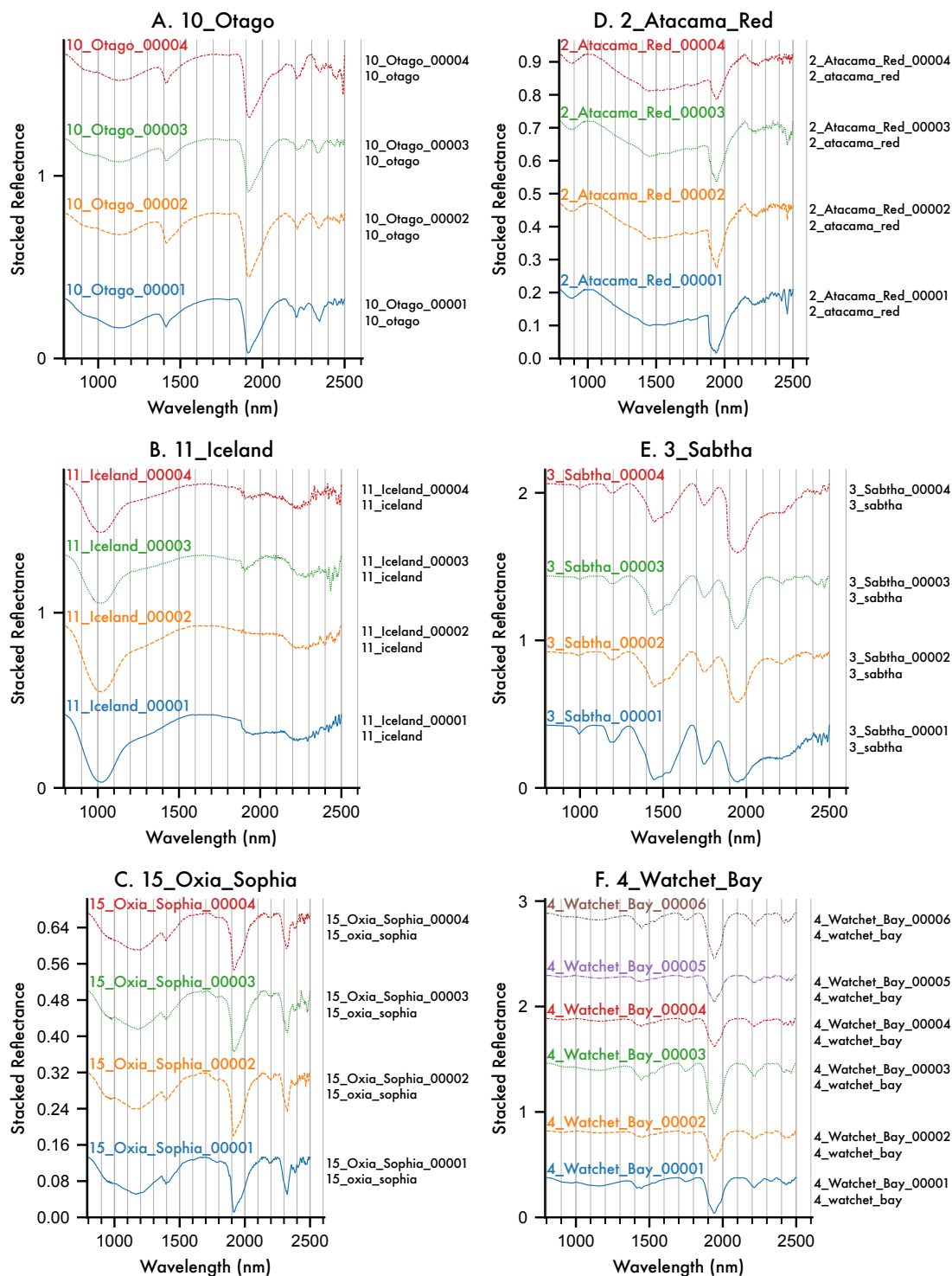
```
[227]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

matcol_sla = sla(matcol)
matcol_cr = matcol_sla.remove_continuum()
```

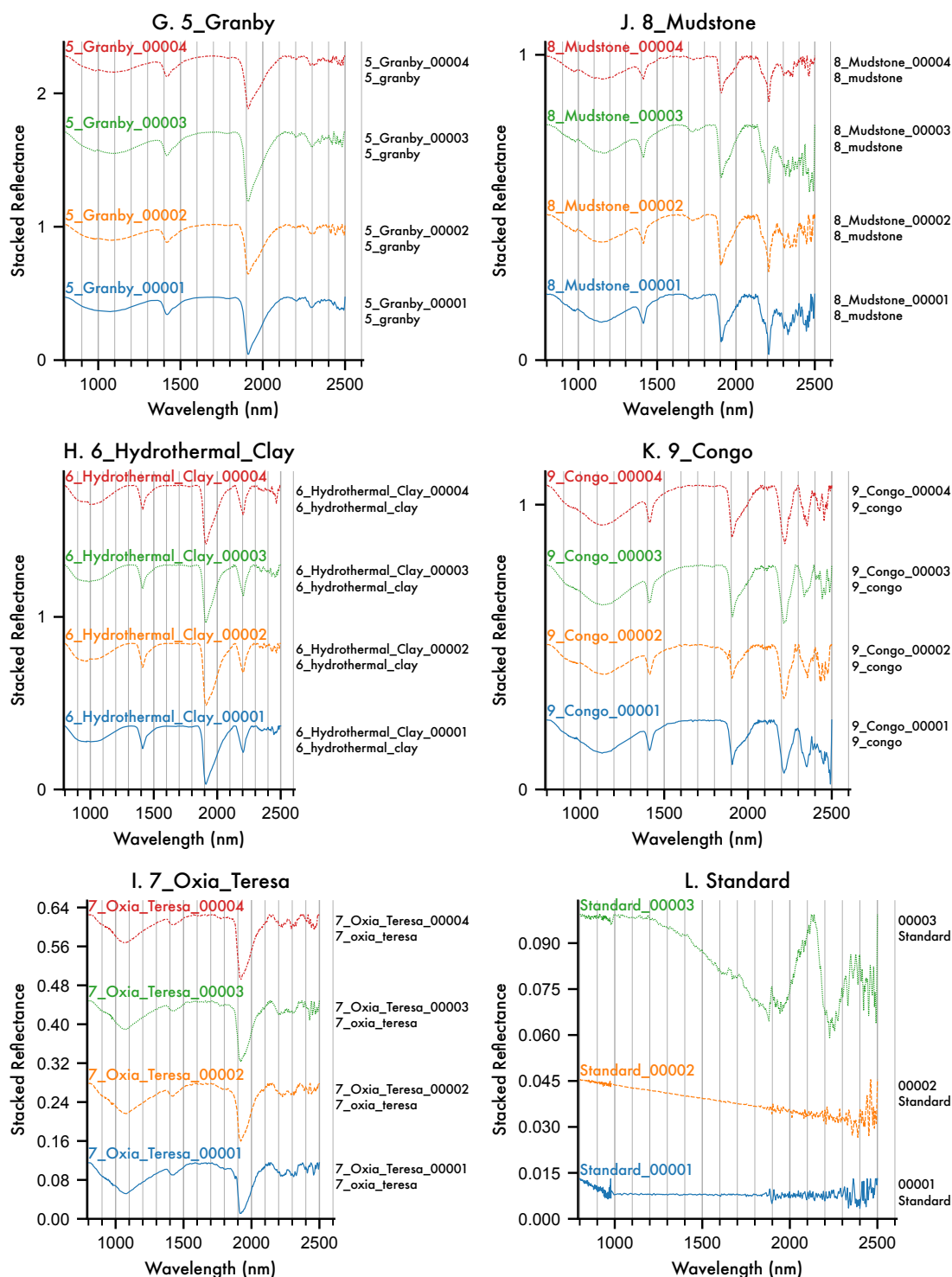
We can plot the profiles to illustrate the continuum removal.

```
[228]: axes = matcol_cr.plot_profiles(stacked=True, scope='species', groupby='Sample_
↳ ID')
```


Species grouped by Sample Id
Continuum Removed (1/3)



Species grouped by Sample Id
Continuum Removed (2/3)



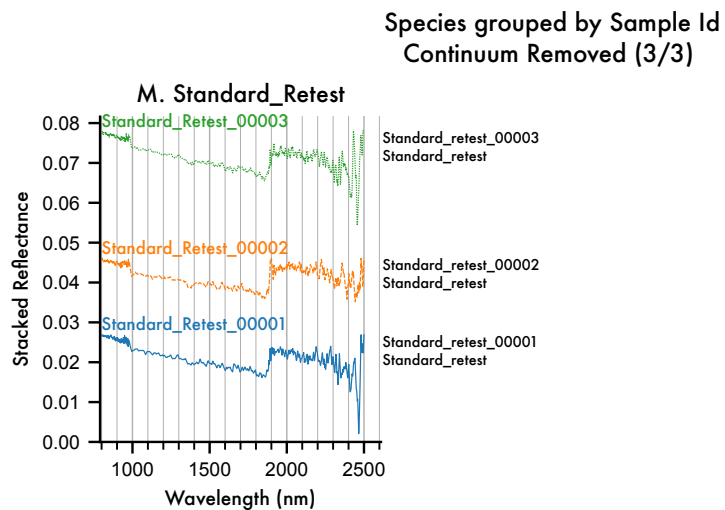
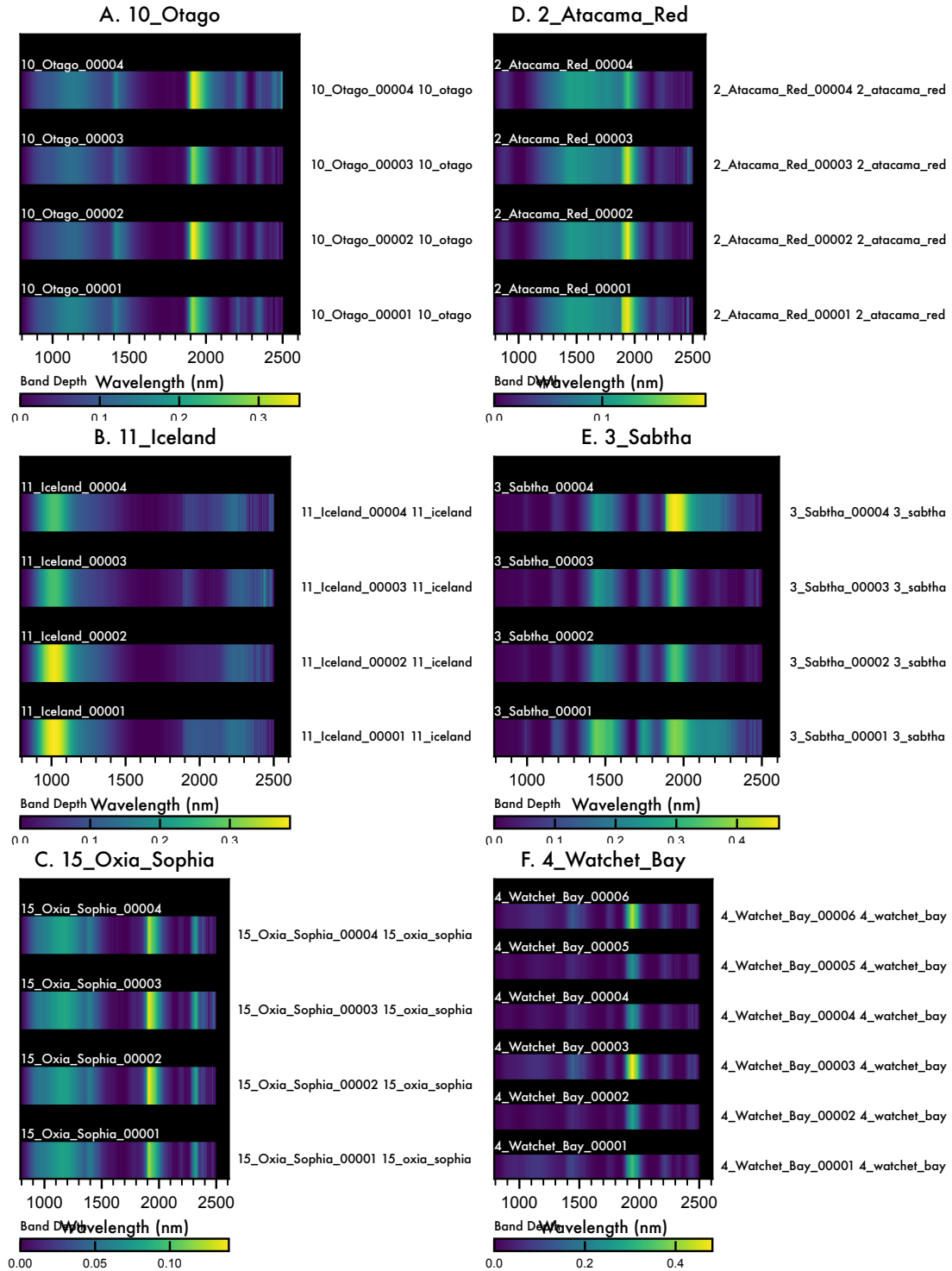


Figure 5 Continuum-removed high resolution MICA Files spectral library, grouped by Category.

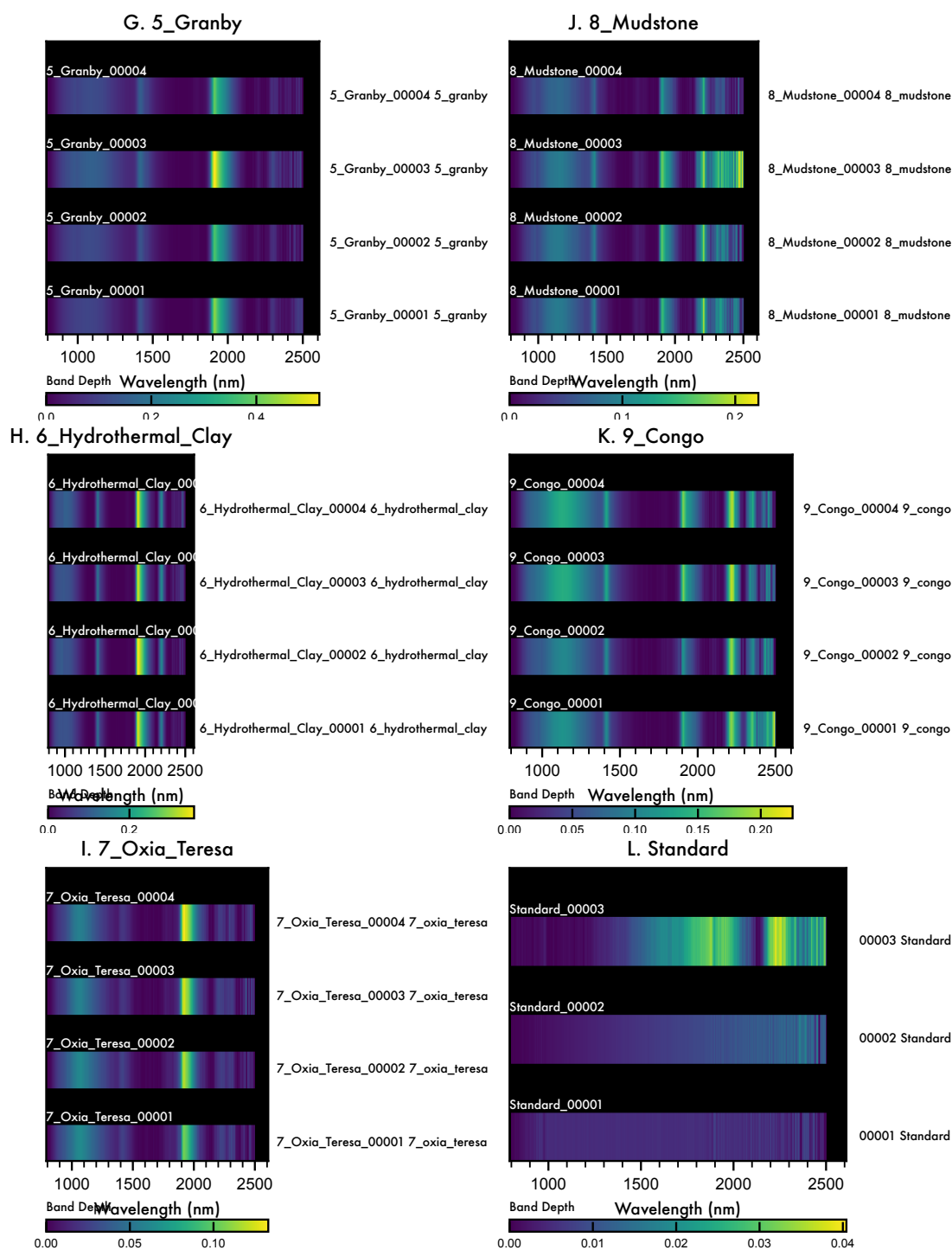
We can illustrate the band locations more clearly with the ‘waterfall’ visualisation:

```
[230]: axes = matcol_cr.plot_profiles(waterfall=True, scope='species', groupby='Sample_ID')
```

Species grouped by Sample Id
Continuum Removed (1/3)



Species grouped by Sample Id
Continuum Removed (2/3)



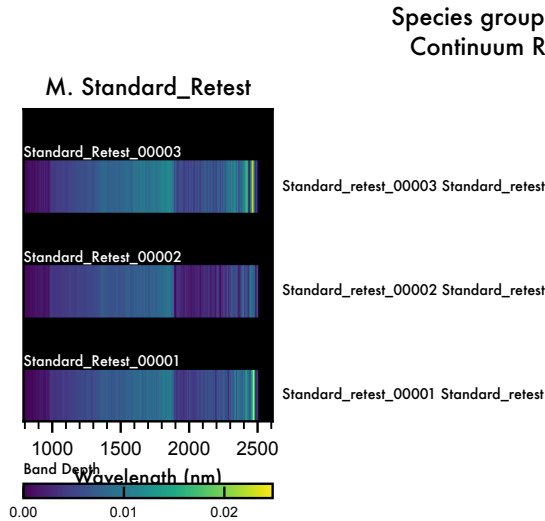


Figure 6 Continuum-removed waterfall plot of high resolution MICA Files spectral library, grouped by Category.

We can see that the waterfall plot represents the band depth information of the `MaterialCollection` in a compact format. Scanning vertically allows for quick identification of common band locations, their depths, widths, and asymmetries.

4.3 Simulating the Enfys Spectral Response

To simulate Enfys, rather than providing a file with the expected spectral response of each position of the Linear-Variable Filter (LVF, above), we specify the spectral range and resolution and sampling regime, and generate a set of central-wavelengths and bandwidths. We do this with the `InstrumentBuilder` class of the `sptk.instrument` module.

```
[231]: from sptk.instrument import InstrumentBuilder
```

4.3.1 Signal-to-Noise Ratio

Previously we loaded the spectral SNR, as extracted from the current version of the Enfys STM. Here we merge the SWIR and MWIR SNR datasets, and prepare the data for passing to the `sptk.Instrument` constructor.

We have adapted the `InstrumentBuilder` function to accept a pandas Series mapping wavelength to SNR. The `InstrumentBuilder` then performs interpolation to the simulated wavelength range.

We splice the SWIR and MWIR data together, using the higher value SNR where the wavelength ranges overlap:

```
[232]: # concatenate the inas_mwir_snr and the ingaas_swir, using the highest SNR for
        ↳ overlapping wavelength values
ingaas_mwir_snr = ingaas_swir_snr.combine_first(ingaas_mwir_snr)
```

We also convert the wavelengths from μm to nm:

```
[233]: enfys_ingaas_mwir_snr.index = enfys_ingaas_mwir_snr.index * 1000
```

for computational convenience, we should set values of $\text{SNR} \leq 3$ to NaN, to interpret these as ‘bad bands’

```
[234]: # where SNR <=1 to NaN, to interpret these as 'bad bands'
import numpy as np
enfys_ingaas_mwir_snr = enfys_ingaas_mwir_snr.where(enfys_ingaas_mwir_snr > 3,
↳ other=np.nan)
```

```
[235]: enfys_ingaas_mwir_snr.index.name = 'wavelength'
```

The continuous spectral SNR for the complete instrument is:

```
[236]: g = enfys_ingaas_mwir_snr.plot(logy=True)
# set the x and y axes
g.set_xlabel('Wavelength (nm)')
g.set_ylabel('SNR')
title = g.set_title('Enfys InGaAs SWIR & InGaAs MWIR Merged Sensor SNR')
```

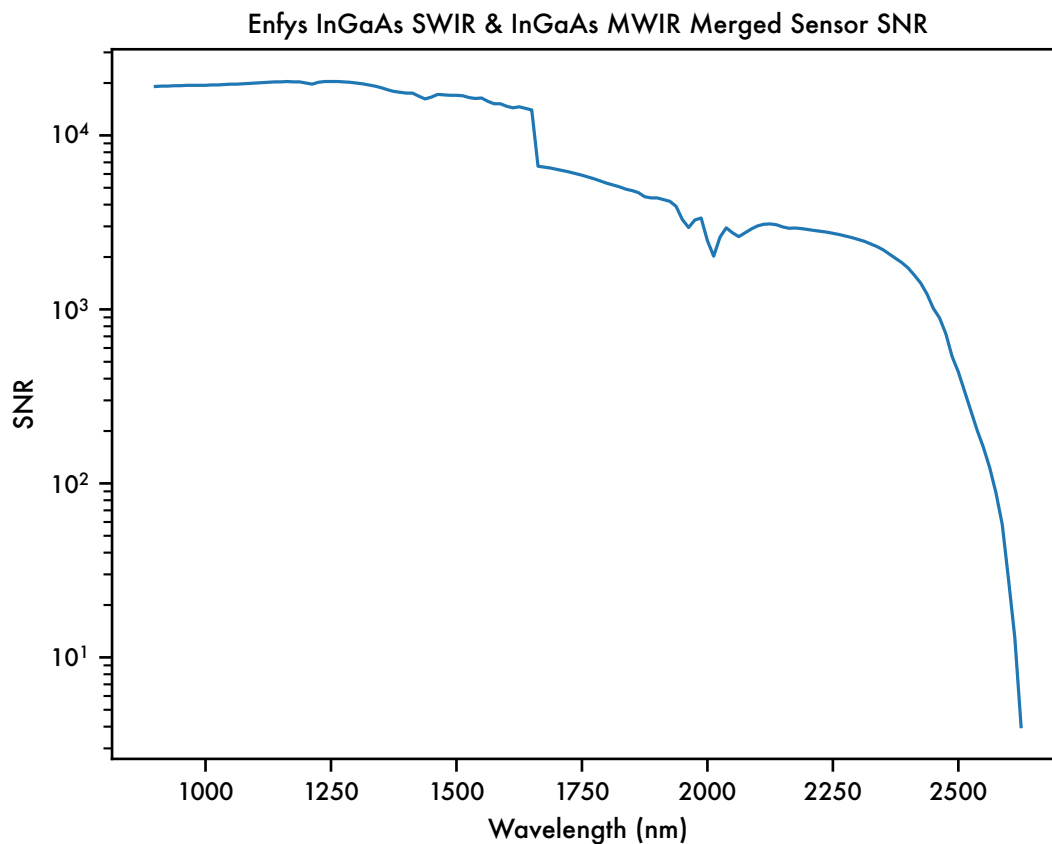


Figure 7 SWIR & MWIR merged spectral Signal-to-Noise Ratio input data, extracted from the current Enfys Science Traceability Matrix, maintained by M. Gunn.

4.3.2 Spectral Resolving Power

Previously we loaded in the latest data for modelling the Spectral Resolving Power for the SWIR and MWIR LVFs. Here we merge the dataset, and prepare it to be passed to the `sptk.Instrument` module.

We merge the datasets, using the SWIR values where the sensor range overlaps.

```
[237]: # get the max wavelength where ingaas_swir_snr is not nan
ingaas_swir_snr = ingaas_swir_snr[~np.isnan(ingaas_swir_snr)]
max_swir_wavelength = ingaas_swir_snr.index[-1] * 1000 # convert from  $\mu\text{m}$  to nm
max_swir_wavelength
# get the enfys_swir_respow data where the wavelength is less than the
    ↪ max_swir_wavelength
enfys_swir_respow = enfys_swir_respow[enfys_swir_respow.index <
    ↪ max_swir_wavelength]
enfys_swir_respow
# get the enfys_mwir_respow where the wavelength is greater than the
    ↪ max_swir_wavelength
enfys_mwir_respow = enfys_mwir_respow[enfys_mwir_respow.index >
    ↪ max_swir_wavelength]
enfys_mwir_respow
# now merge the datasets
enfys_respow = pd.concat([enfys_swir_respow, enfys_mwir_respow])
```

The complete instrument Spectral Resolving Power is:

```
[238]: g=enfys_respow.plot(marker='o',xlabel='Wavelength (nm)', ylabel=r'$\lambda/$
    ↪ $\Delta\lambda$ Spectral Resolving Power')
g.legend(['SWIR & MWIR'])
title = g.set_title('Enfys SWIR & MWIR merged Spectral Resolving Power
    ↪ $\gamma[\lambda]$')
```

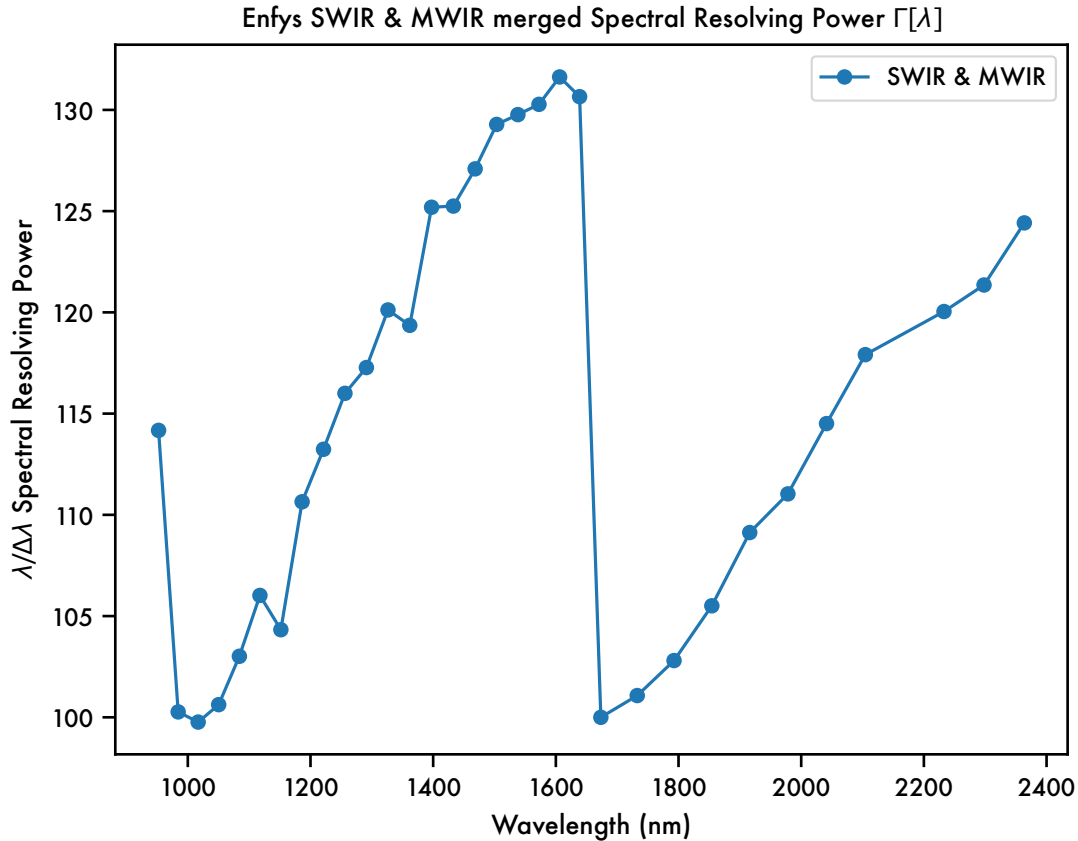



Figure 8 SWIR and MWIR merged Spectral Resolving Power data, as extracted from the Enfys schedule review 20250807.ppt, M. Gunn.

We pass this data to the InstrumentBuilder function.

4.3.3 Instrument Build

Now we will pass this to the InstrumentBuilder, defining the spectral range as $\lambda \in [900, 3100]$ nm.

```
[239]: enfys_builder = InstrumentBuilder(
    instrument_name='enfys_ingaas_mwir',
    instrument_type='lvf',
    sampling='nyquist',
    resolution = enfys_respow,
    spectral_range=[900, 2501],
    snr = enfys_ingaas_mwir_snr)
```

Exporting instrument to ../data/instruments/enfys_ingaas_mwir.csv...

```
[240]: from sptk.instrument import Instrument
```

Now we build the transmission profiles for each LVF position, according to the list of CWLs

and FWHMs generated above. These are accessed by reading the .csv file produced by the InstrumentBuilder procedure.

When generating the profiles, we turn the CWL and FWHM into a profile by assuming a distribution function. For Enfys we use the Cauchy distribution, under the guidance of the Enfys build team, which behaves similarly to a Gaussian, but with wider ‘wings’. It is parameterised by the full-width-at-half-maximum, as the distribution strictly has no finite variance. We have implemented a build_cauchy_filter method for the Instrument class.

```
[241]: enfys = Instrument(  
    name = 'enfys_ingaas_mwir', # filename of the instrument,  
    ↪information, held in /software/data/instruments/  
    project_name = matcol.project_name, # the project directory name,  
    ↪hosting outputs in the /spectral_parameter_studies/ directory  
    shape = 'cauchy',  
    load_existing=False, # if True, load existing instrument data from,  
    ↪the project directory  
    plot_profiles=False, # if true, plot the transmission profiles,  
    ↪during construction  
    export_df=True) # if True, export the instrument transmission  
    ↪profiles to the project directory
```

Building Instrument...

Building new DataFrame for 'enfys_ingaas_mwir'...

Exporting the Instrument to CSV and Pickle formats...

```
[242]: fig ,ax = enfys.plot_instrument_characteristics()
```

Plotting Instrument Transmission...

```
/opt/homebrew/Caskroom/miniconda/base/envs/enfys_mission_reference_samples/lib/p  
ython3.10/site-packages/seaborn/_oldcore.py:200: FutureWarning: elementwise  
comparison failed; returning scalar instead, but in the future will perform  
elementwise comparison
```

```
    if palette in QUAL_PALETTES:
```

Plotting Maximum Signal-to-Noise Ratio...

Plotting FWHM...

Plotting Spectral Resolution...

Plots exported to ../projects/enfys_mission_reference_samples/instrument

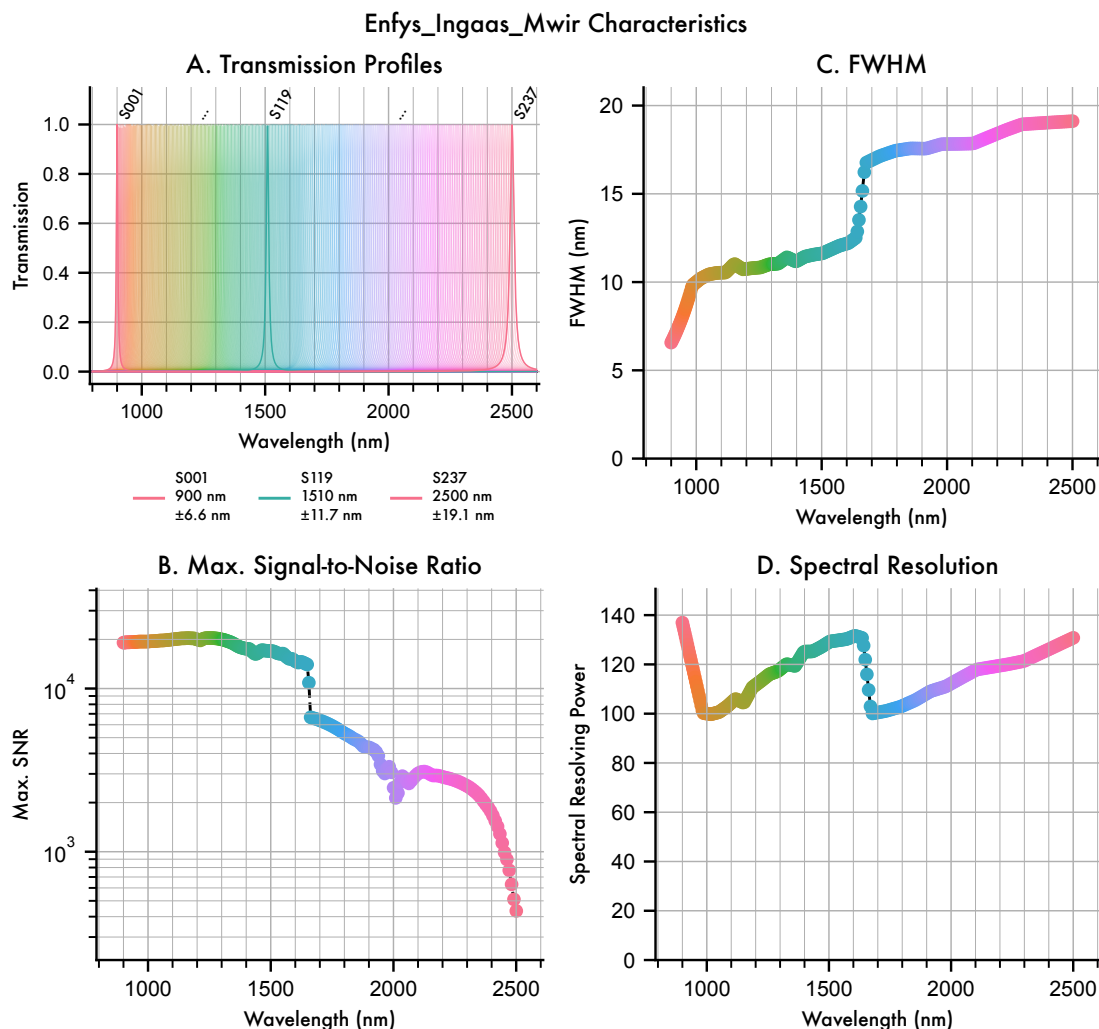


Figure 9 Simulated spectral response and resolution of *Enfys*, based on preliminary expectations of the spectral and radiometric response.

The data shown in figure 9 approximates the expected performance of *Enfys*. This data represents the current component level understanding of the filter and sensor performance of the built instrument.

5 Results

5.1 Sampling with Enfys

We now make an `Observation` object by sampling the MICA files `MaterialCollection` with the `Enfys Instrument`.

```
[254]: from sptk.observation import Observation
```

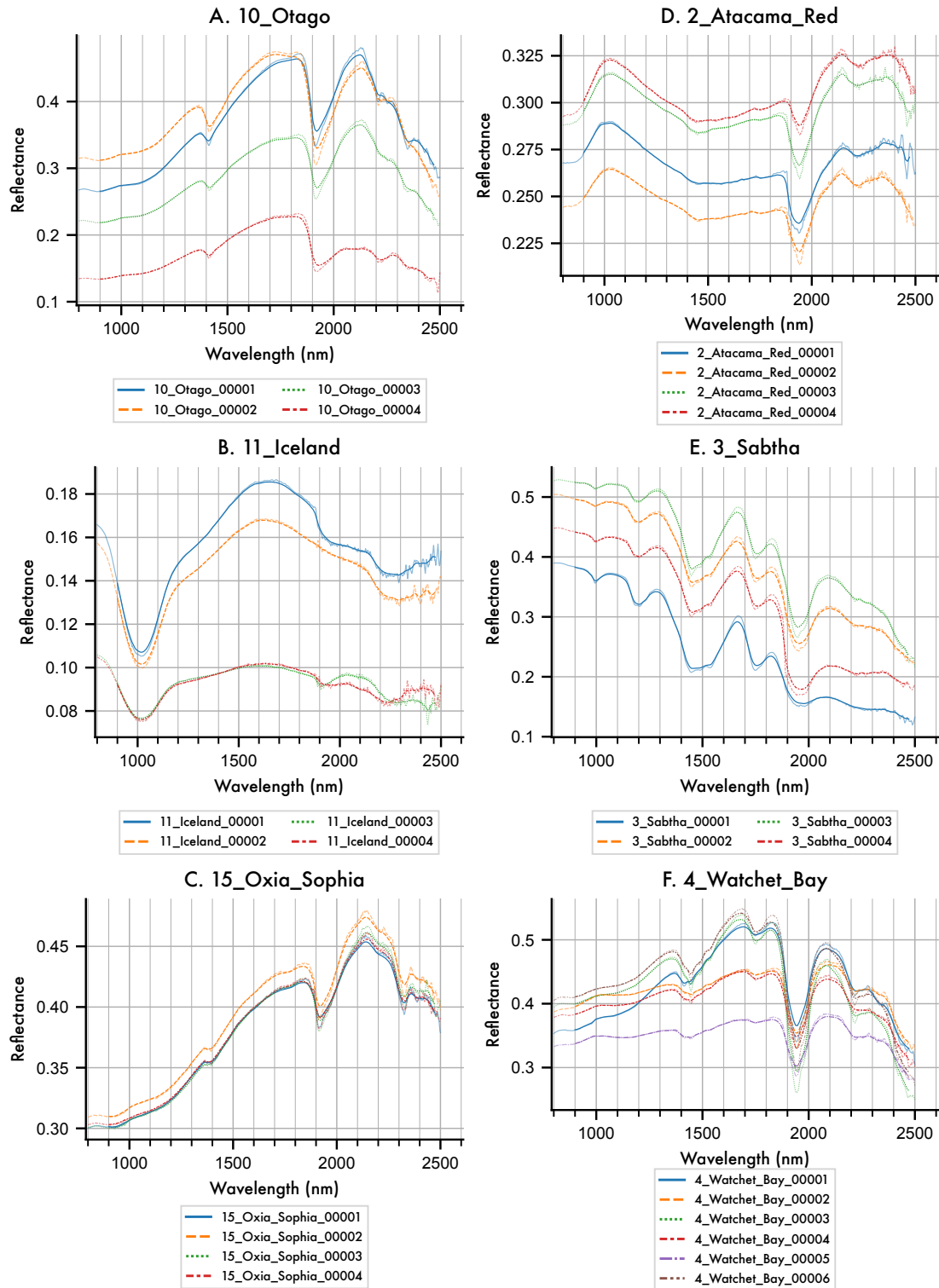
```
[255]: obs = Observation(  
    material_collection=matcol,  
    instrument = enfys,  
    load_existing=False,  
    plot_profiles=False,  
    export_df=True)
```

Building new Observation DataFrame
for 'enfys_mission_reference_samples'...
Sampling the Material Collection with the Instrument...
Sampling complete.
Exporting the Observation Pickle format...
Observation export complete.

We can plot the sampled spectra against the input high-spectra below:

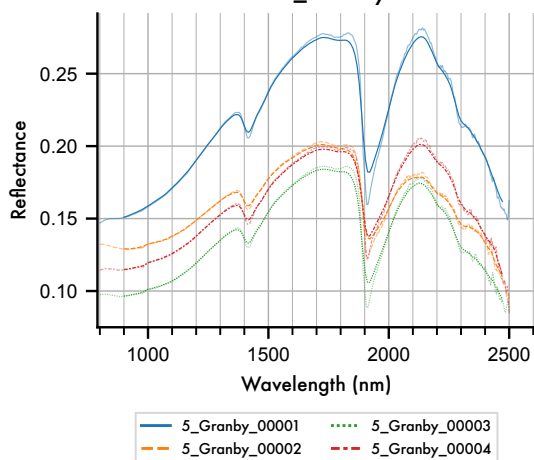
```
[246]: # axes = obs.plot_profiles(scope='categories', groupby='Subgroup',  
    ↪stacked=True, hires_under=True)  
  
axes = obs.plot_profiles(stacked=False, scope='species', groupby='Sample ID',  
    ↪hires_under=True)
```

Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id (1/3)

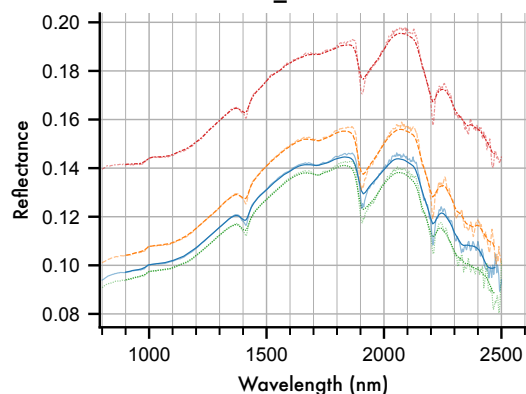


Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id (2/3)

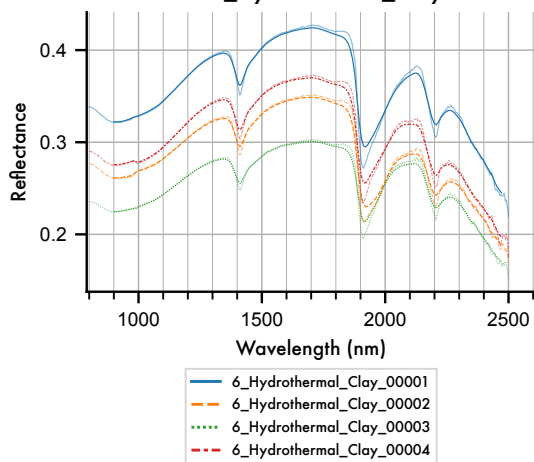
G. 5_Granby



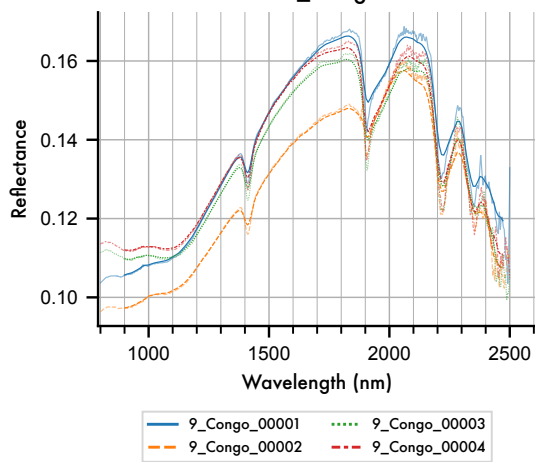
J. 8_Mudstone



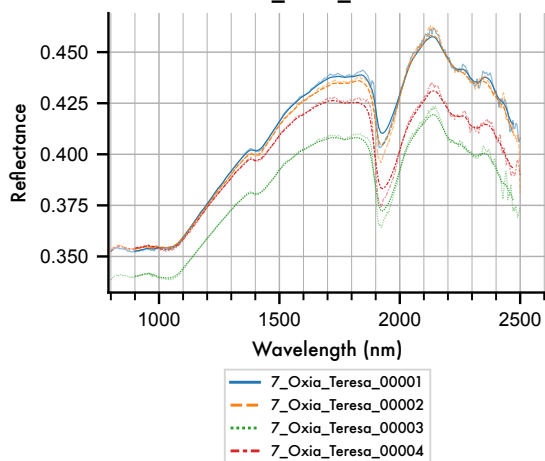
H. 6_Hydrothermal_Clay



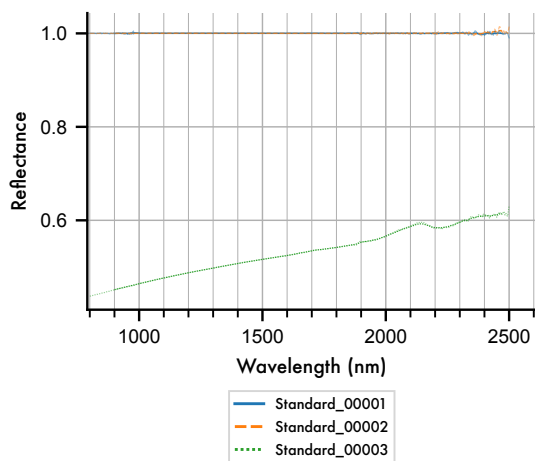
K. 9_Congo



I. 7_Oxia_Teresa



L. Standard



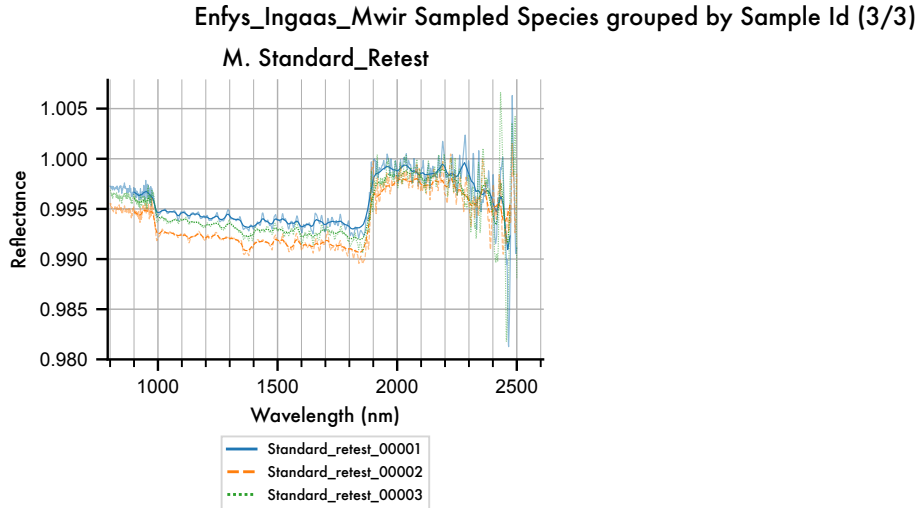


Figure 10 The MICA files as sampled by *Enfys*, with the original high-resolution spectra plotted underneath each.

```
[247]: axes = obs.plot_profiles(scope='8_Mudstone', groupby='Sample ID',
    ↪stacked=False, hires_under=True)
```

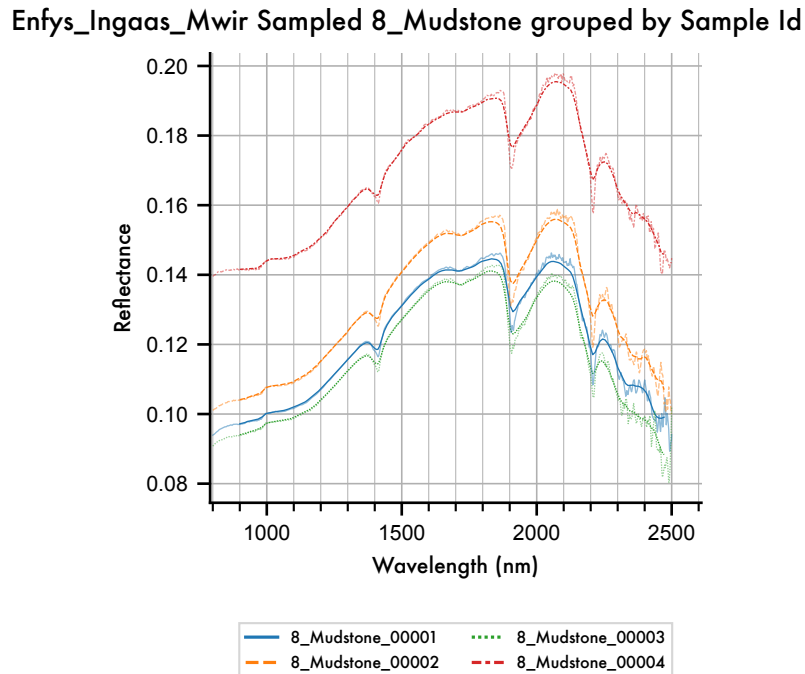


Figure 11 Deviation of the *Enfys* sampled spectrum from the high-resolution input spectrum for

Illite.

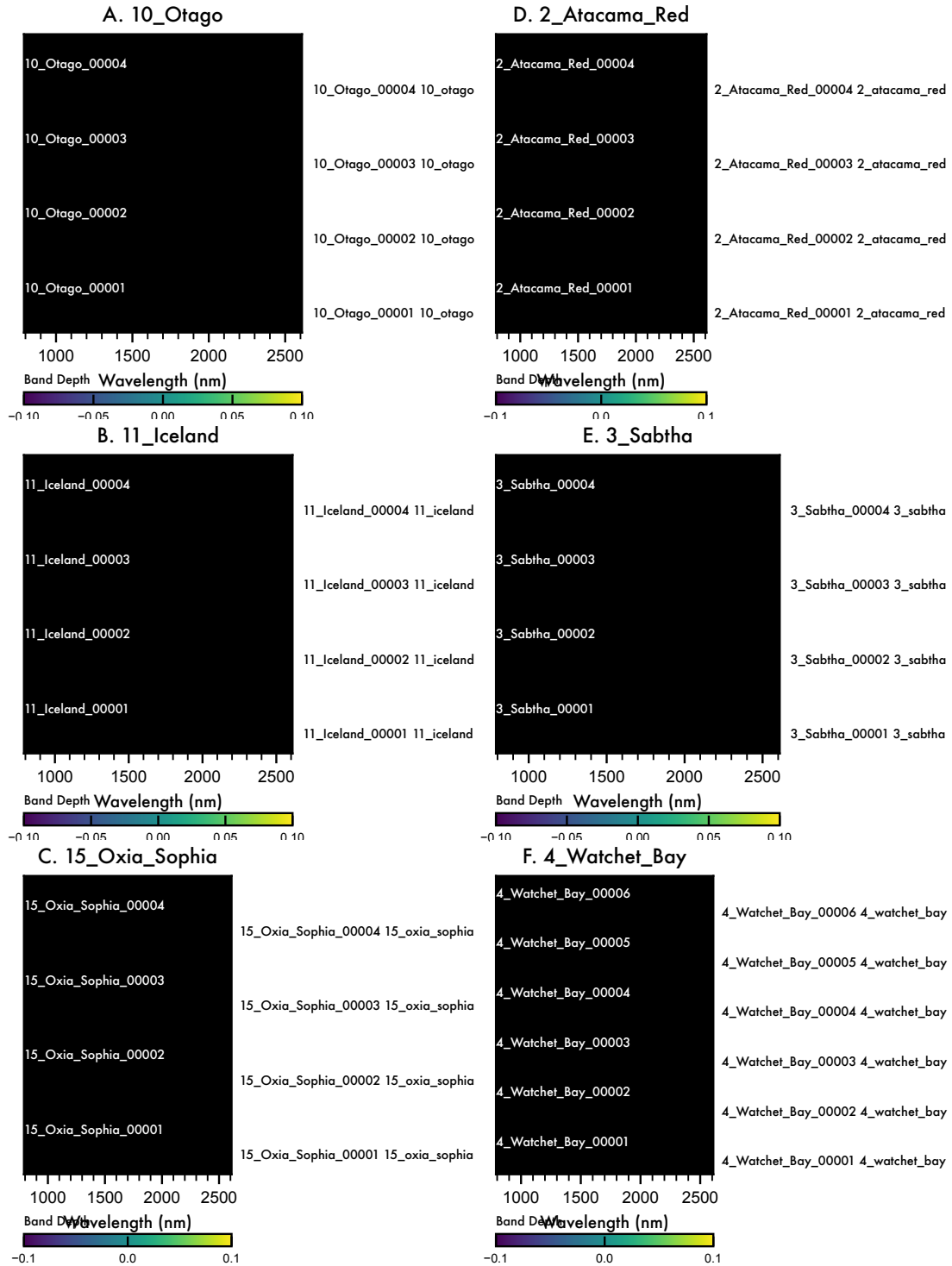
We note that there is an effect by which peaks and troughs in the spectra are reduced. This is due to the long tails of the Cauchy filter profiles. Despite the reduction of absolute band depth, the features of Illite (fig. 8) are still resolvable.

We can produce a continuum-removed waterfall plot to check that the major bands are resolved.

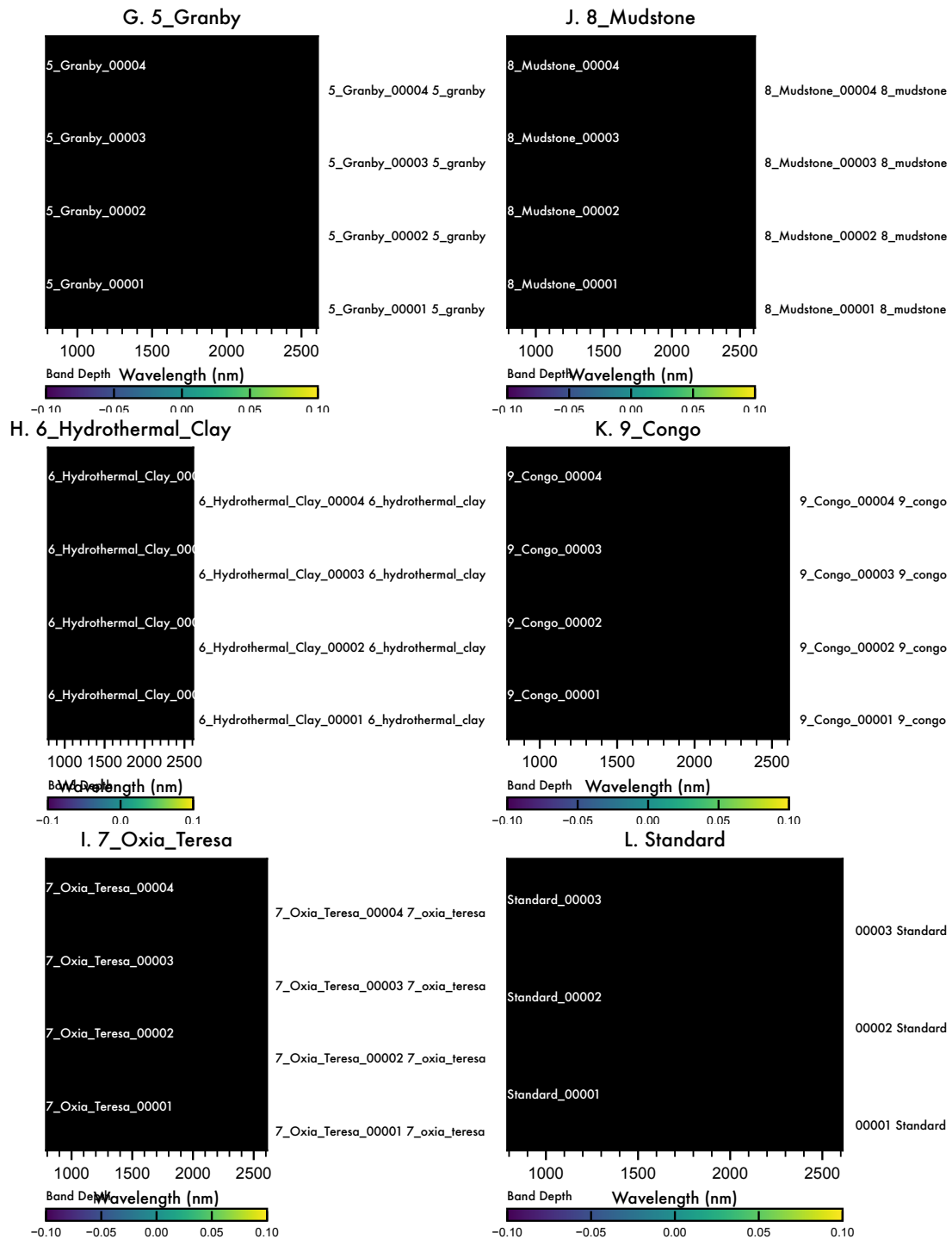
```
[250]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla  
  
obs_cr = sla(obs).remove_continuum()
```

```
[249]: # axes = obs_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')  
  
axes = obs_cr.plot_profiles(waterfall=True, scope='species', groupby='Sample_  
↳ ID')
```


Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id
Continuum Removed (1/3)



Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id
Continuum Removed (2/3)



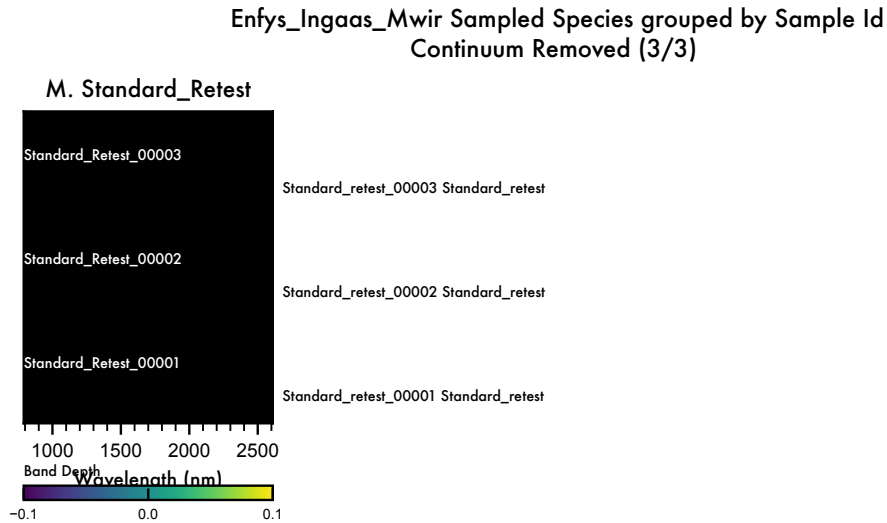


Figure 12 Continuum-Removed waterfall plot of the MICA files as sampled by *Enfys*.

Comparing Figure 12 to Figure 6, we can see that the weaker sub-2 μm bands are less well resolved.

We can save this data

```
[256]: obs_cr.export_main_df()
```

Exporting the Observation Pickle format...

Observation export complete.

5.2 Adding Synthetic Noise

We have the capability to add synthetic noise to the dataset, by redrawing a sample from the dataset with noise added according to the signal-to-noise ratio specified for the given channel.

We can optionally redraw multiple noisy samples for each entry, to give an indication of the statistical uncertainty, or for example draw a single noisy sample, to produce a single-observation mission-realistic dataset.

First, let's draw a single noisy entry:

```
[257]: noise_obs = obs.add_noise(1, seed=0)
```

```
[258]: axes = obs.plot_profiles(scope='8_Mudstone', groupby='Sample ID',
    ↪stacked=False, hires_under=True, ci=False)
```

Enfys_Ingaas_Mwir Sampled 8_Mudstone grouped by Sample Id with Noise

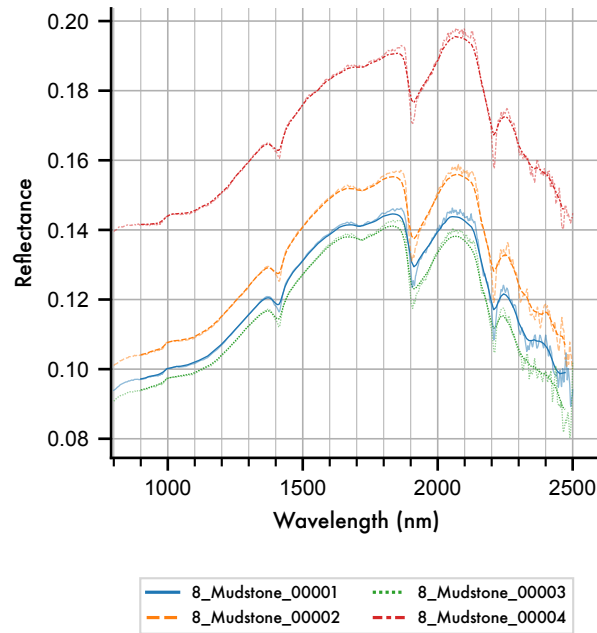


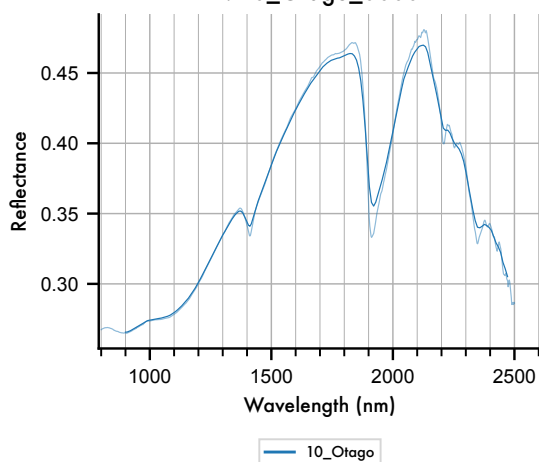
Figure 13 Noisy resampled example spectrum for Illite, showing 1 noisy sample according to the spectral signal-to-noise ratio illustrated in figure 9.B.

We can illustrate this in greater detail by plotting each entry on a separate figure, with the y-axis clipped to the spectral range of entry. Note that for samples with a small spectral range (i.e. samples that are spectrally flat), this stretching will appear to amplify noise, so the y-axis range must be considered when assessing the noise.

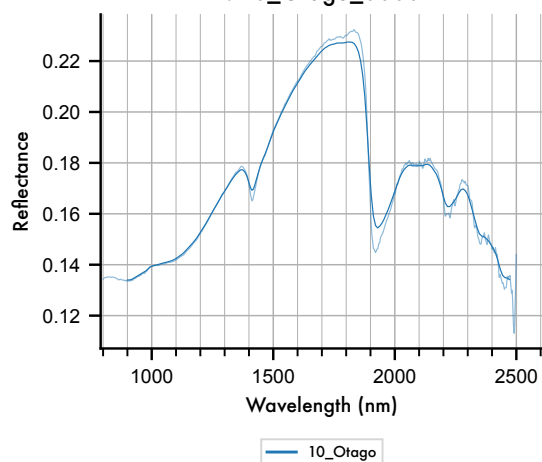
```
[259]: axes = obs.plot_profiles(scope='samples', groupby='Species', stacked=False,
    ↪ hires_under=True, ci=False)
```

Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (1/9)

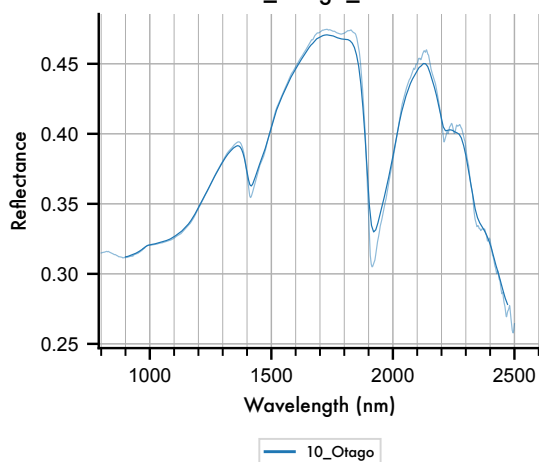
A. 10_Otago_00001



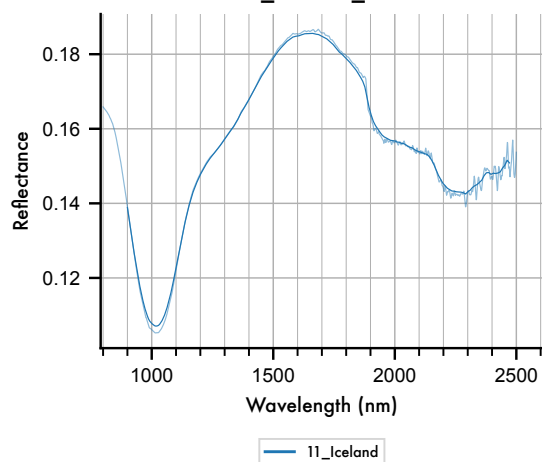
D. 10_Otago_00004



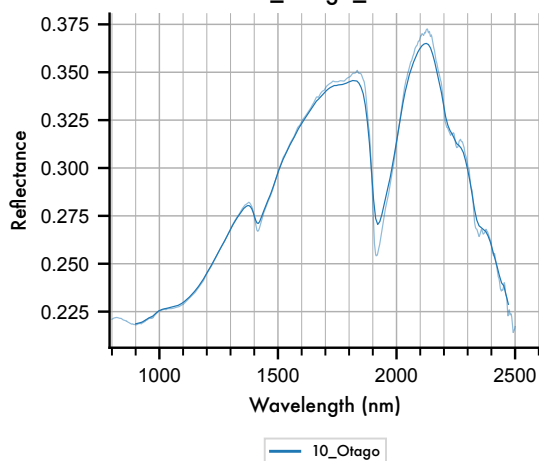
B. 10_Otago_00002



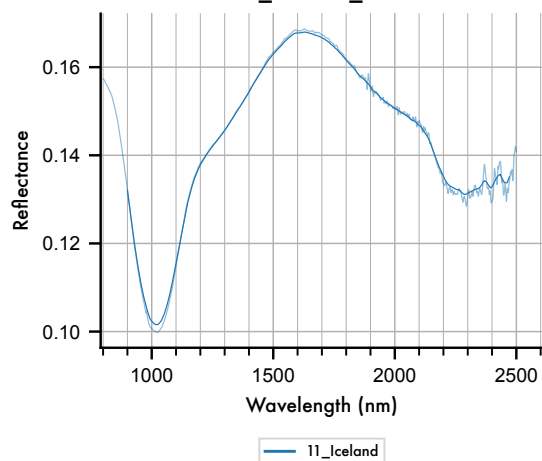
E. 11_Iceland_00001



C. 10_Otago_00003

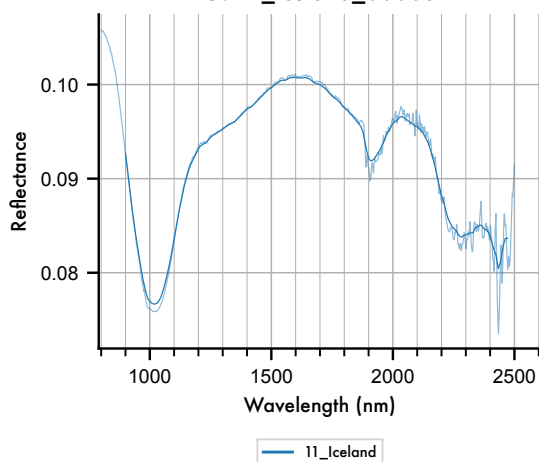


F. 11_Iceland_00002

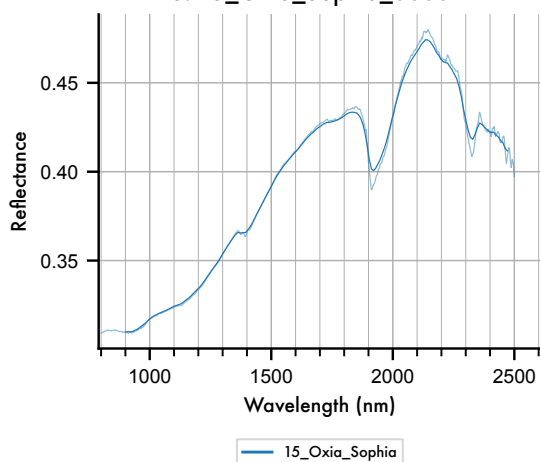


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (2/9)

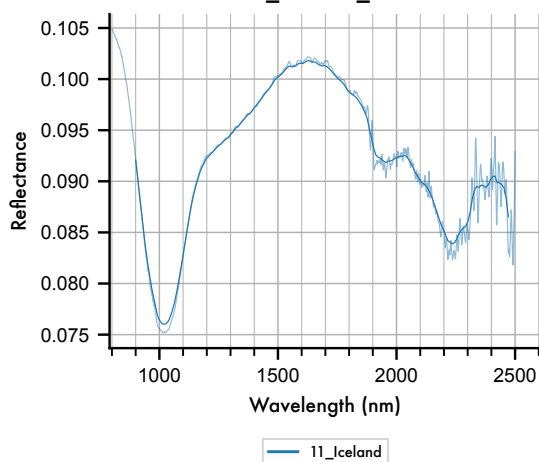
G. 11_Iceland_00003



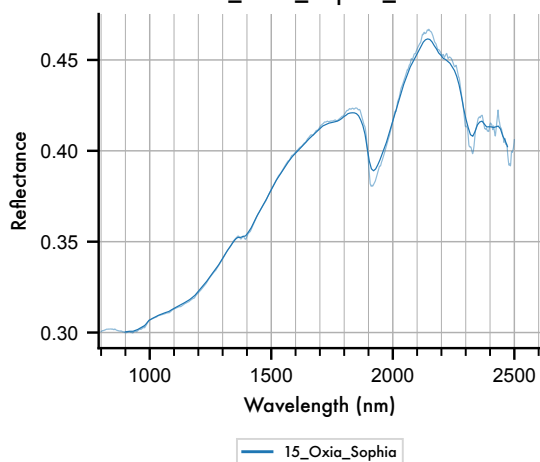
J. 15_Oxia_Sophia_00002



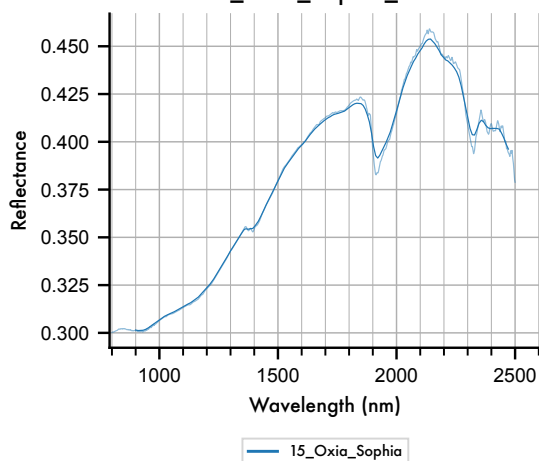
H. 11_Iceland_00004



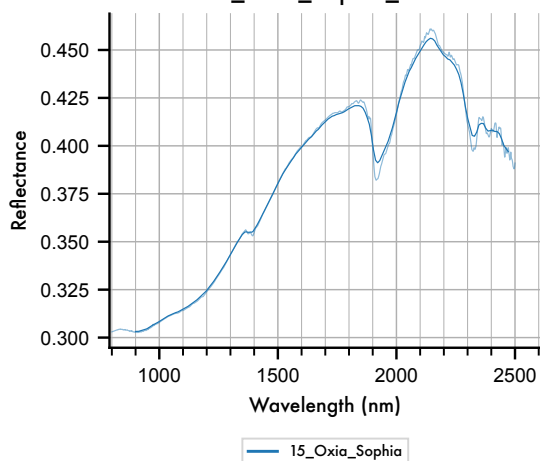
K. 15_Oxia_Sophia_00003



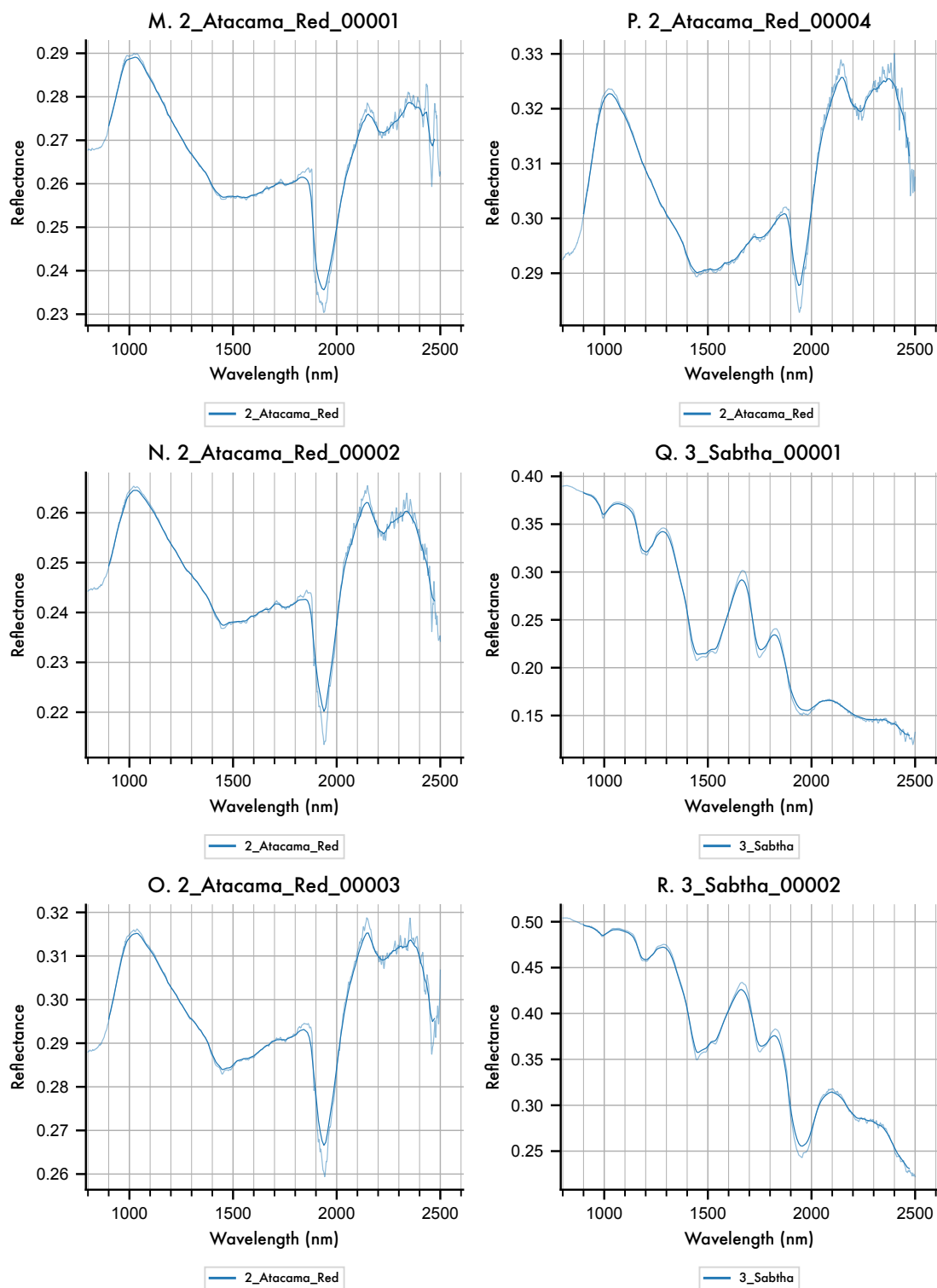
I. 15_Oxia_Sophia_00001



L. 15_Oxia_Sophia_00004

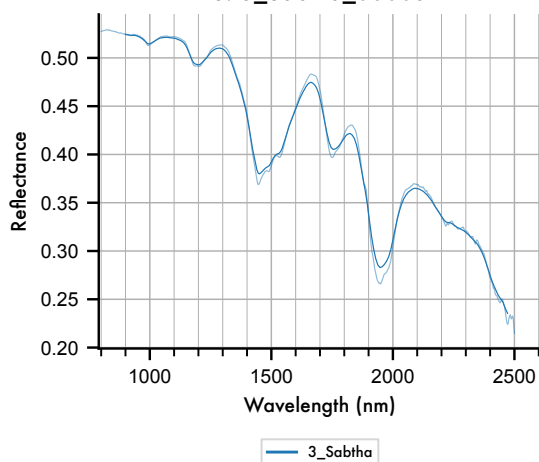


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (3/9)

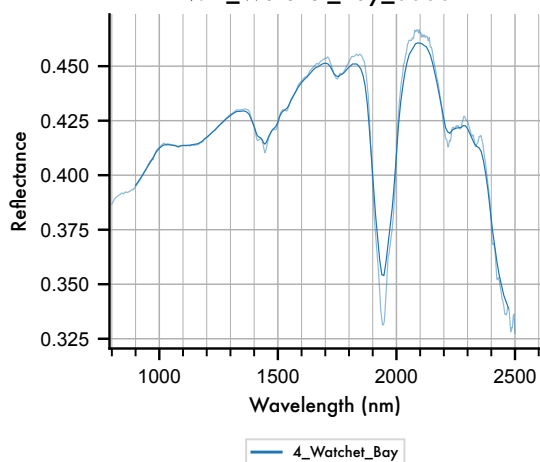


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (4/9)

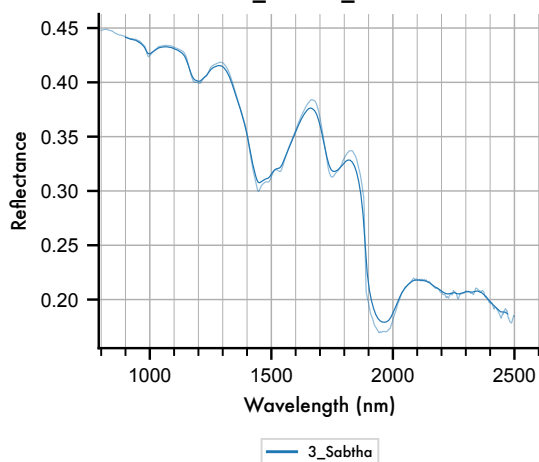
S. 3_Sabtha_00003



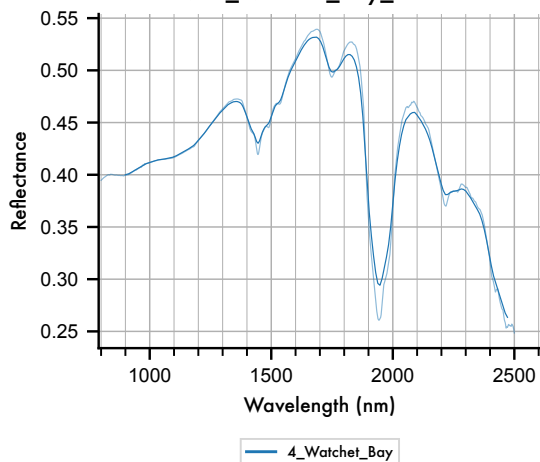
V. 4_Watchet_Bay_00002



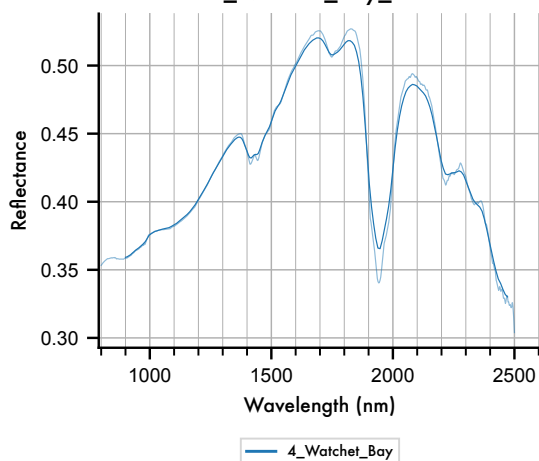
T. 3_Sabtha_00004



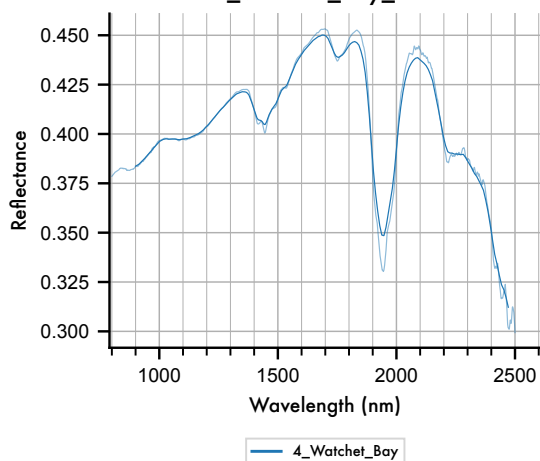
W. 4_Watchet_Bay_00003



U. 4_Watchet_Bay_00001

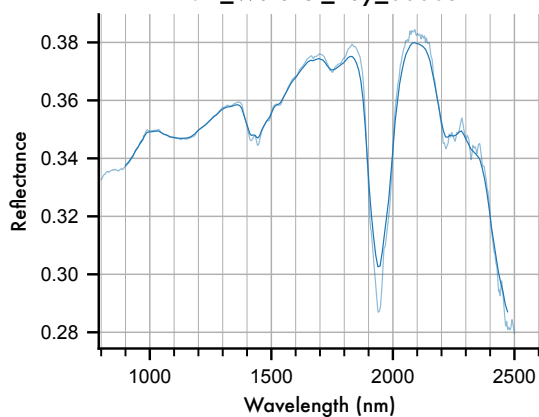


X. 4_Watchet_Bay_00004



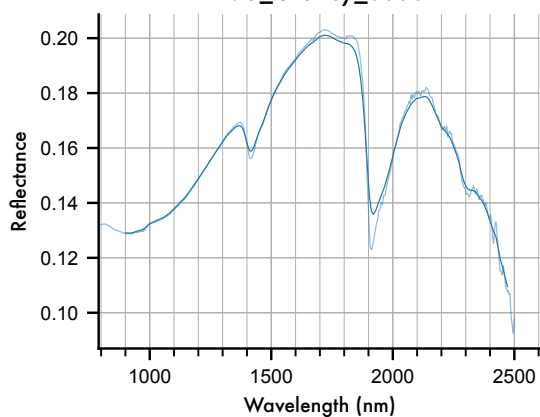
Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (5/9)

Y. 4_Watchet_Bay_00005



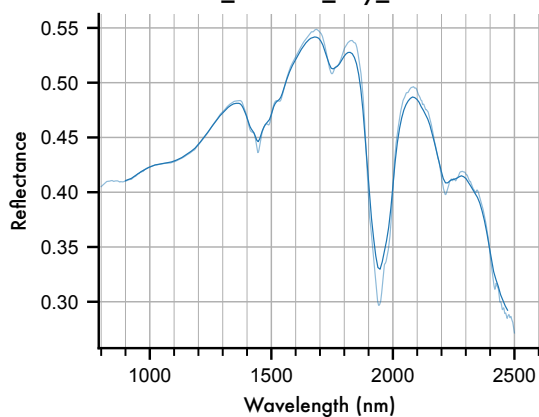
4_Watchet_Bay

AB. 5_Granby_00002



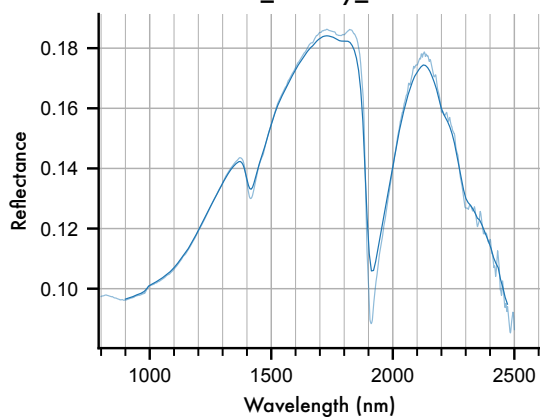
5_Granby

Z. 4_Watchet_Bay_00006



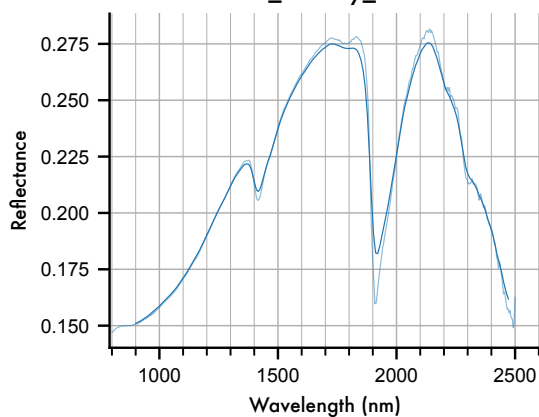
4_Watchet_Bay

AC. 5_Granby_00003



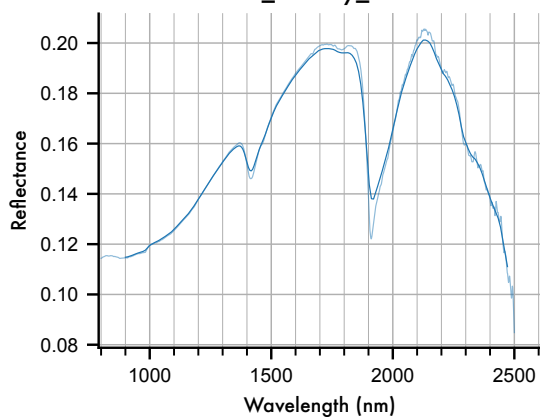
5_Granby

AA. 5_Granby_00001



5_Granby

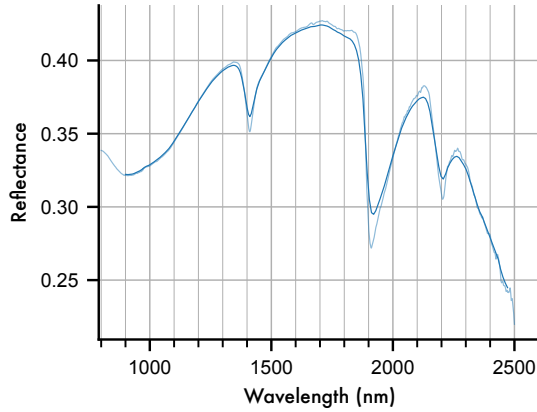
AD. 5_Granby_00004



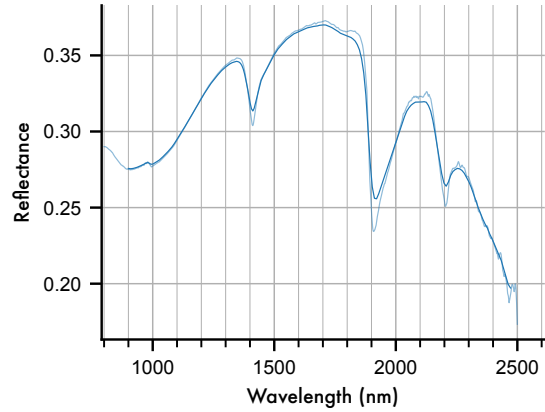
5_Granby

Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (6/9)

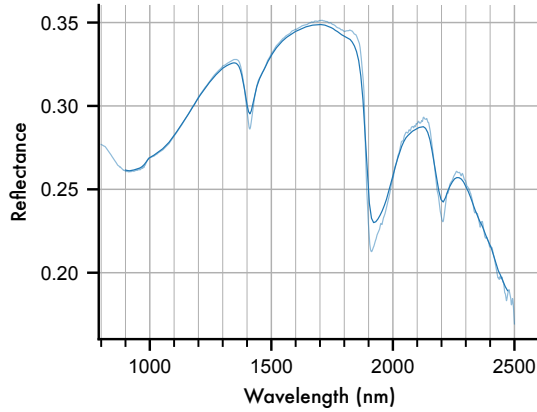
AE. 6_Hydrothermal_Clay_00001



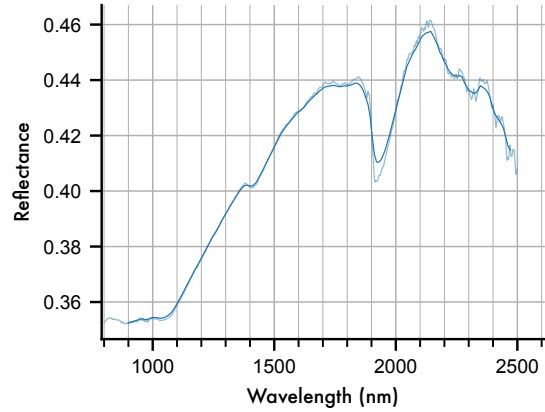
AH. 6_Hydrothermal_Clay_00004



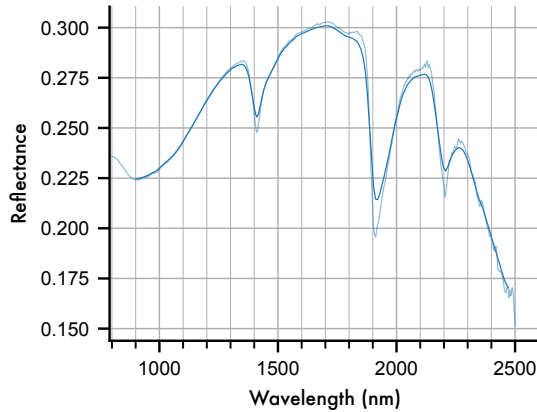
AF. 6_Hydrothermal_Clay_00002



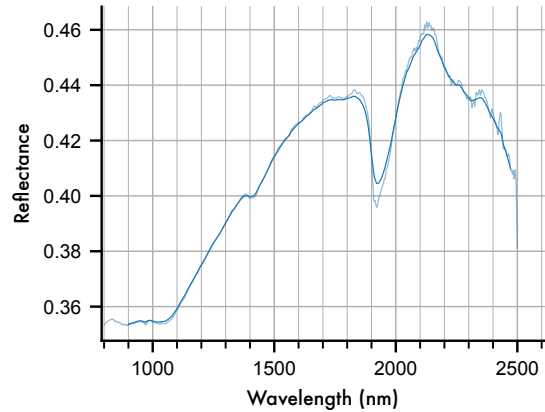
AI. 7_Oxia_Teresa_00001



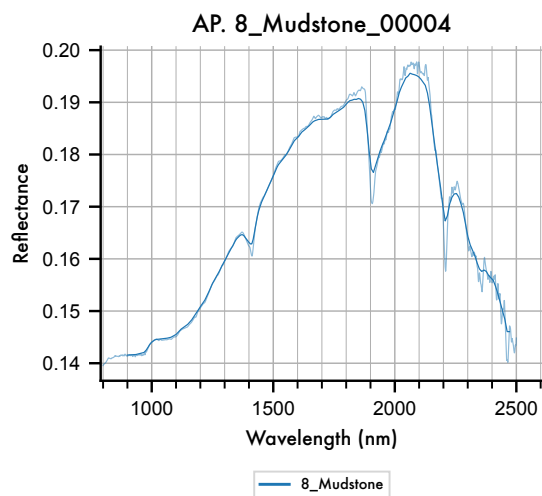
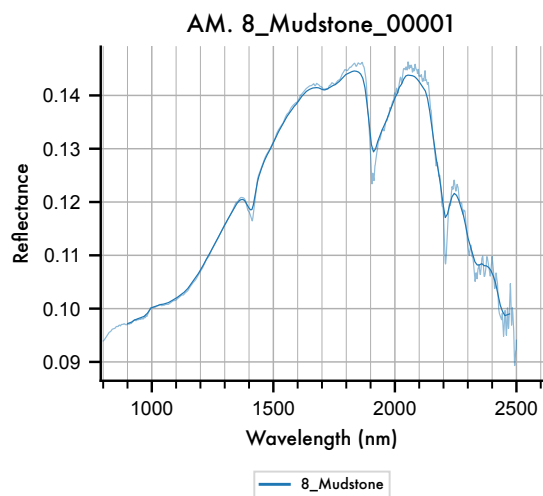
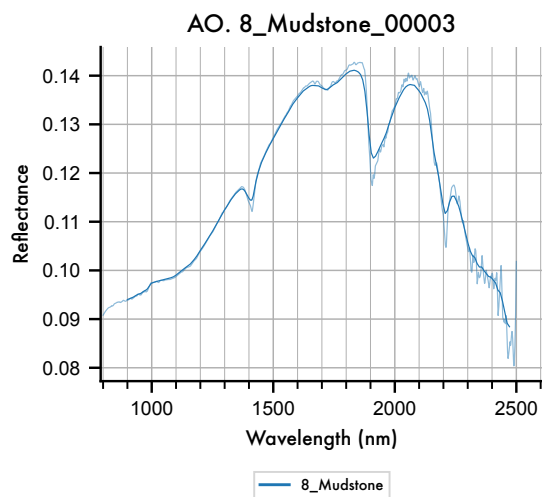
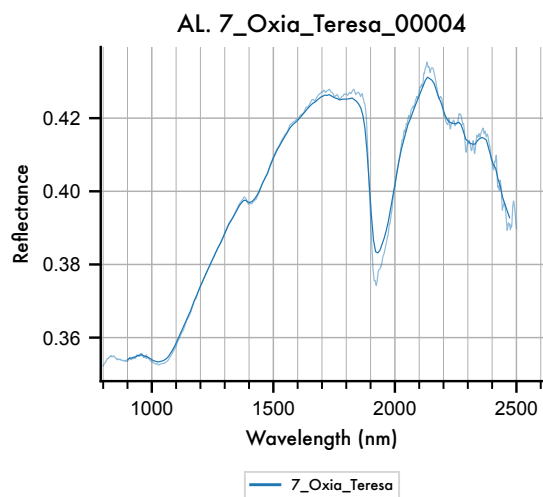
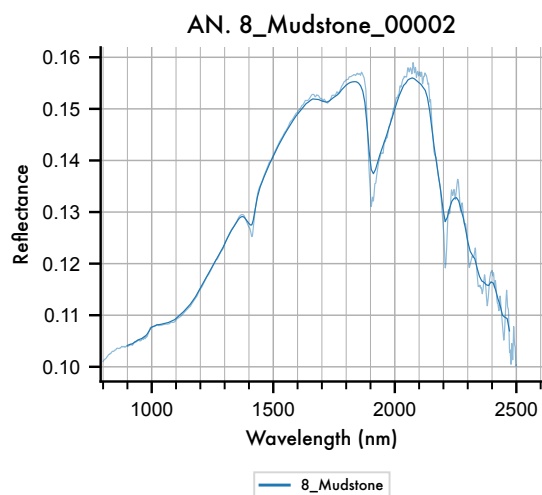
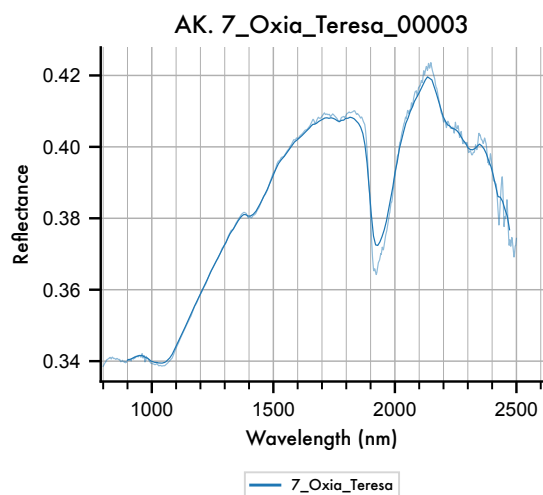
AG. 6_Hydrothermal_Clay_00003



AJ. 7_Oxia_Teresa_00002

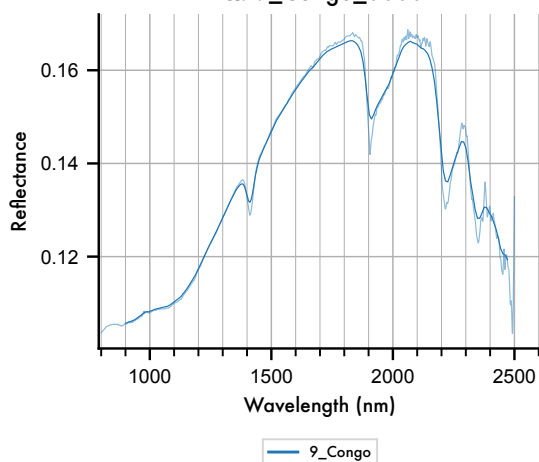


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (7/9)

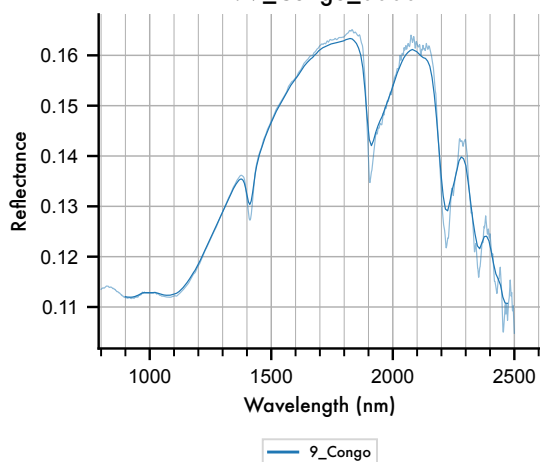


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (8/9)

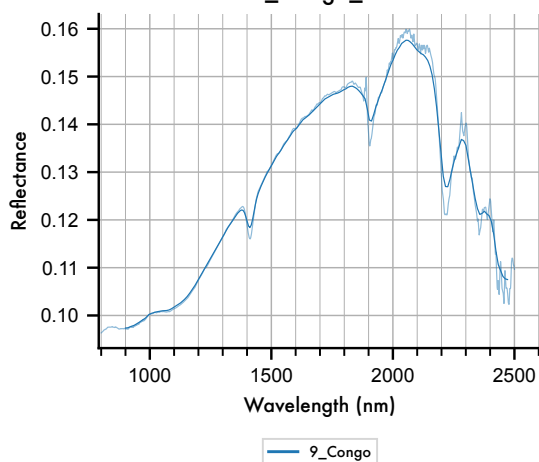
AQ. 9_Congo_00001



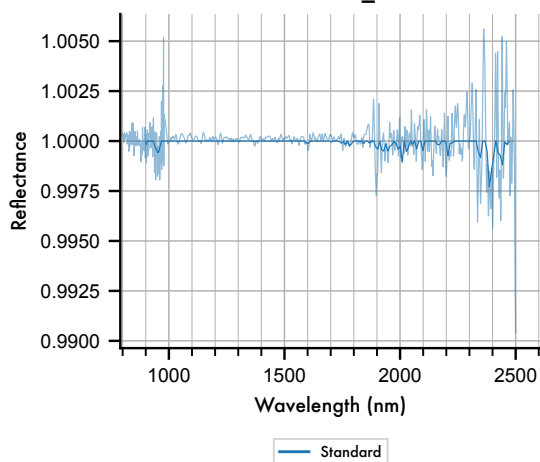
AT. 9_Congo_00004



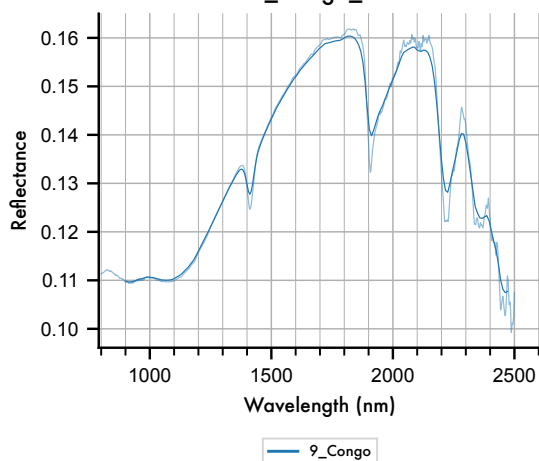
AR. 9_Congo_00002



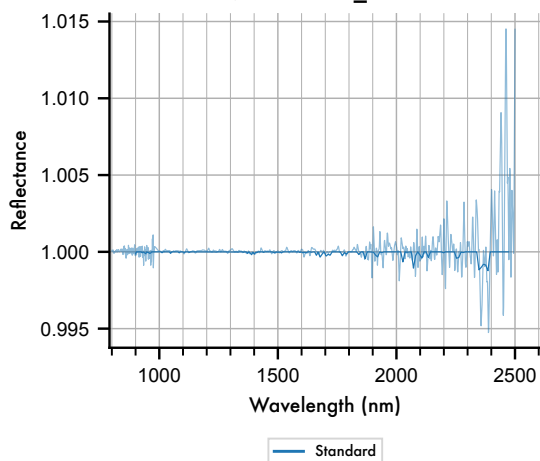
AU. Standard_00001



AS. 9_Congo_00003



AV. Standard_00002



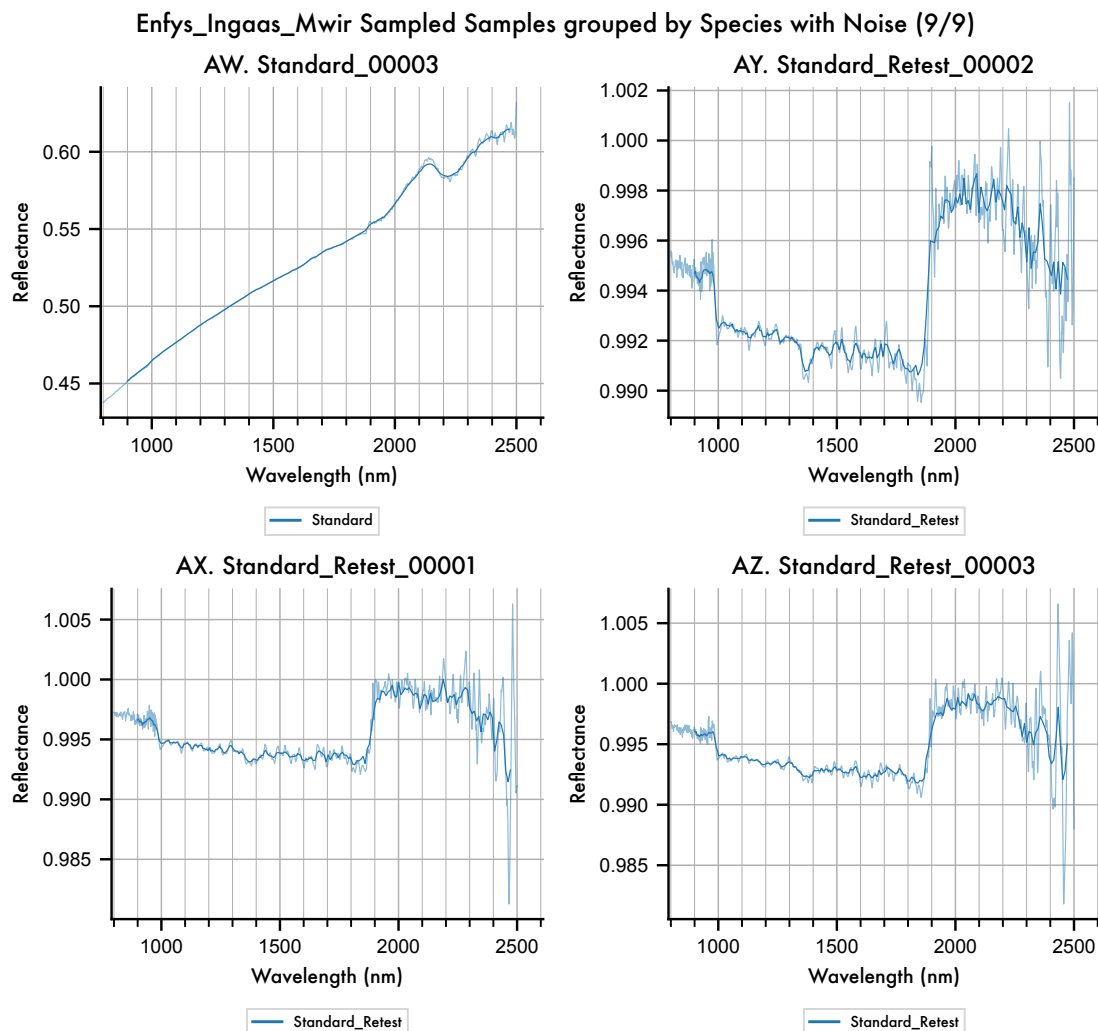


Figure 14 Detailed view of each entry in the spectral library, showing the scale of noise introduced, relative to the features of each spectrum, and the reflectance range.

Qualitatively, these plots illustrate that *Enfys* does not significantly intrude on the diagnostic features of these type specimen of minerals identified on the Mars surface. In some cases, we can see that doublet features are lost, but this is due to the spectral resolving power, rather than the additive noise. We will assess this in the discussion section.

We can export the dataset:

```
[260]: obs.export_main_df()
```

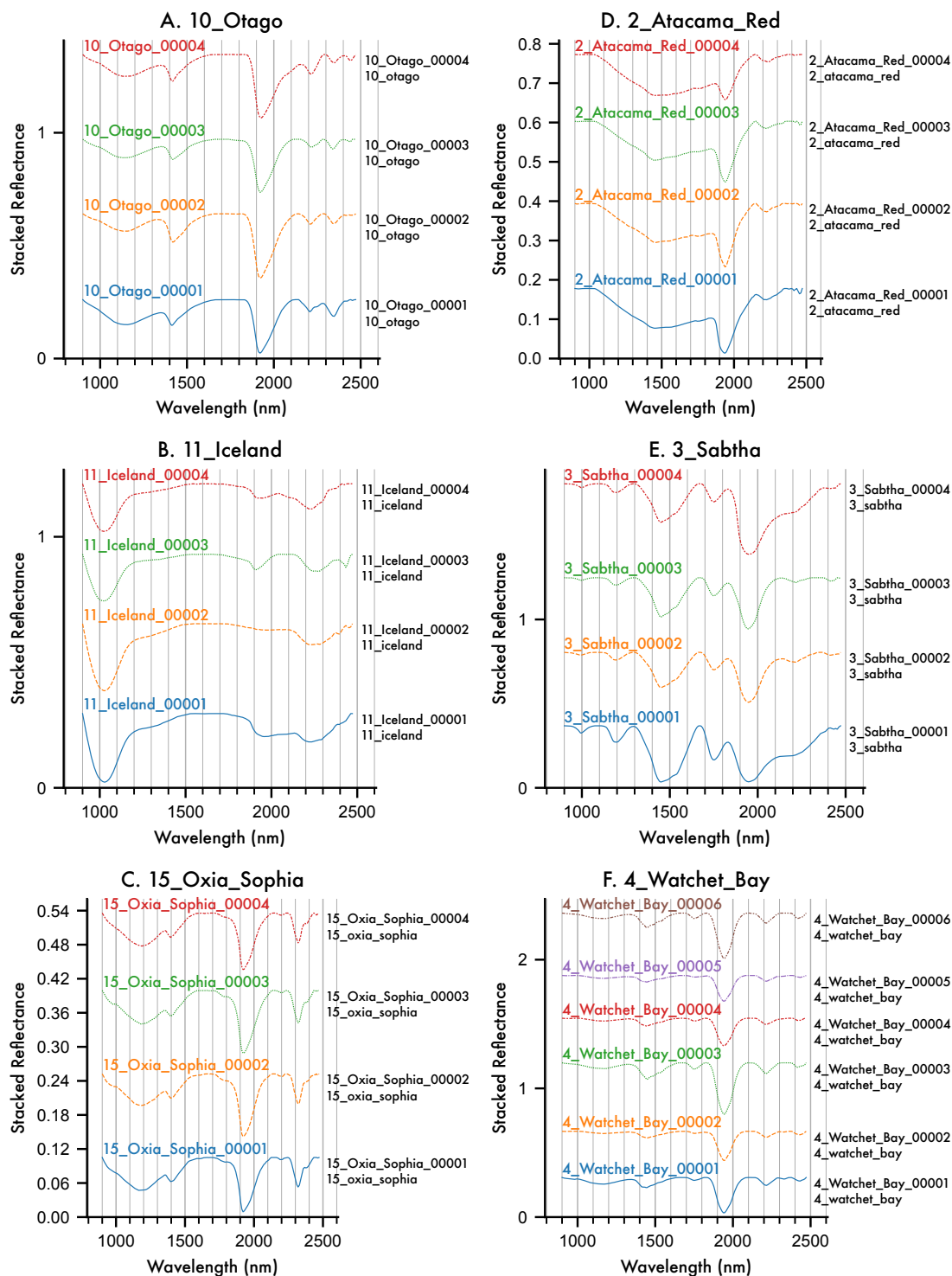
Exporting the Observation Pickle format...
Observation export complete.

5.3 Continuum Removal

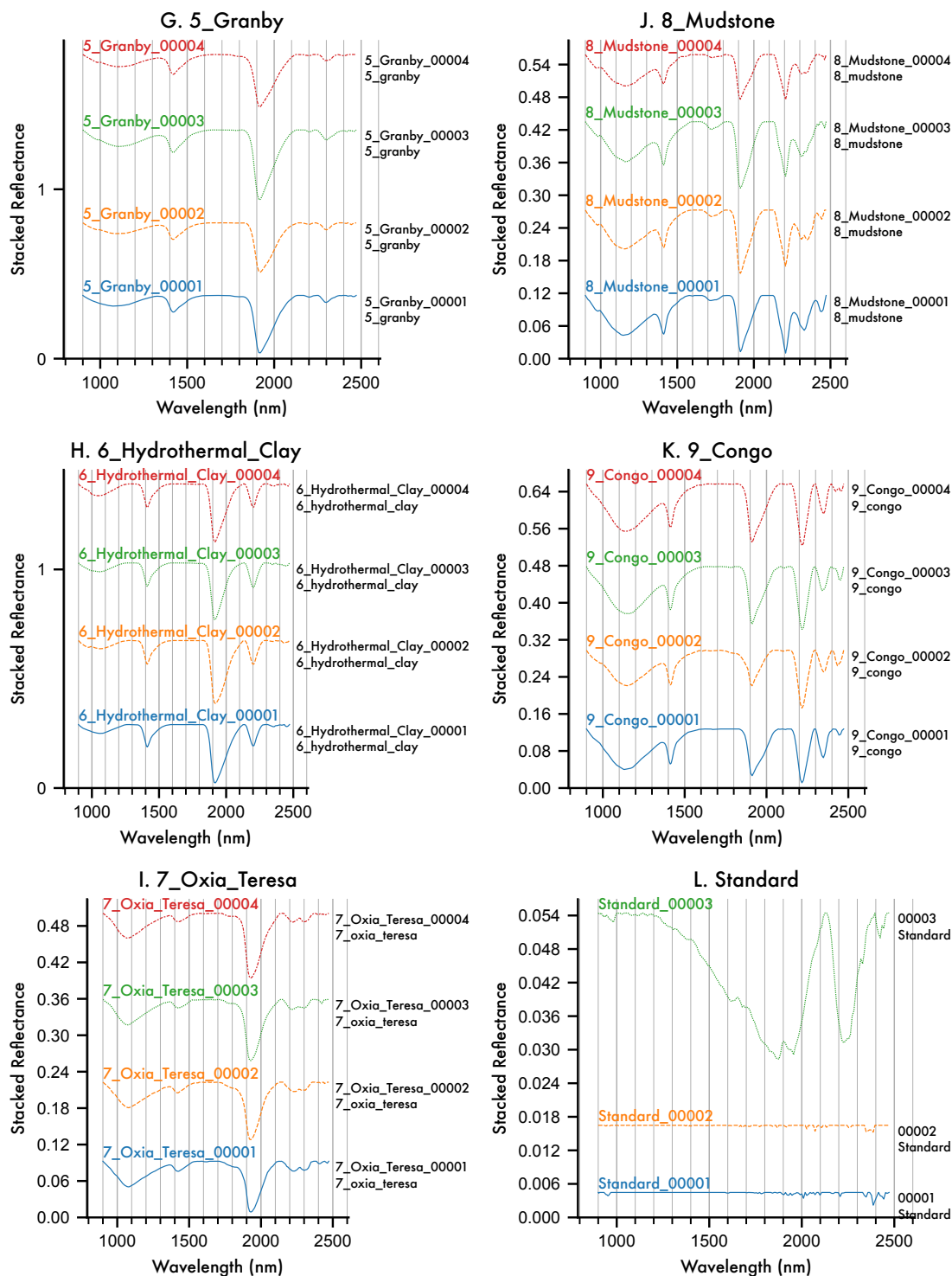
We can apply continuum removal, even to the noisy data.

```
[261]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla  
  
       cr_tool = sla(obs)  
       cr_obs = cr_tool.remove_continuum()  
  
[262]: axes = cr_obs.plot_profiles(scope='species', groupby='Sample ID', stacked=True,  
                                   ↪ hires_under=False, ci=False)
```

Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id
Continuum Removed with Noise (1/3)



Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id
Continuum Removed with Noise (2/3)



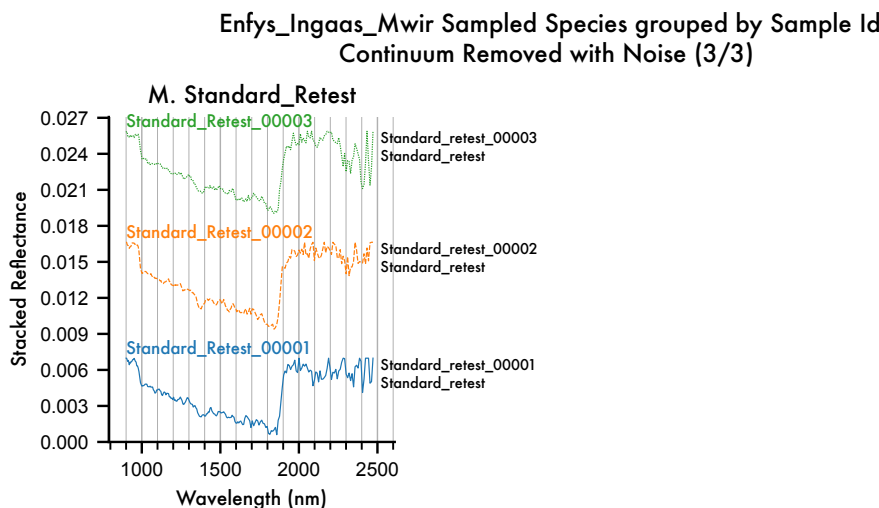


Figure 15 Continuum-Removed entries of the MICA Files, as sampled by *Enfys*, including synthetic noise.

Again, we can export this:

```
[263]: cr_obs.export_main_df()
```

Exporting the Observation Pickle format...
Observation export complete.

5.4 Repeat Noisy Samples: Std. Dev. Confidence Intervals, Continuum-Removal, and Waterfall Visualisation

5.4.1 Repeat Noisy Samples

There are several visualisations that we can produce that demonstrate the average signal over multiple repeatedly drawn noisy spectra. We can simulate this by setting the number of repeats to a larger number, e.g. 64.

```
[264]: _ = obs.add_noise(64, seed=0)
```

Now, we can plot the average spectrum for each entry, over the repeat noisy samples, with shading indicating the standard deviation of the noisy data, where the 'ci' keyword has been applied ('ci': confidence-interval).

```
[266]: axes = obs.plot_profiles(scope='8_Mudstone', groupby='Sample ID',
    ↪stacked=False, hires_under=True, ci=True)
```

Enfys_Ingaas_Mwir Sampled 8_Mudstone grouped by Sample Id with Noise

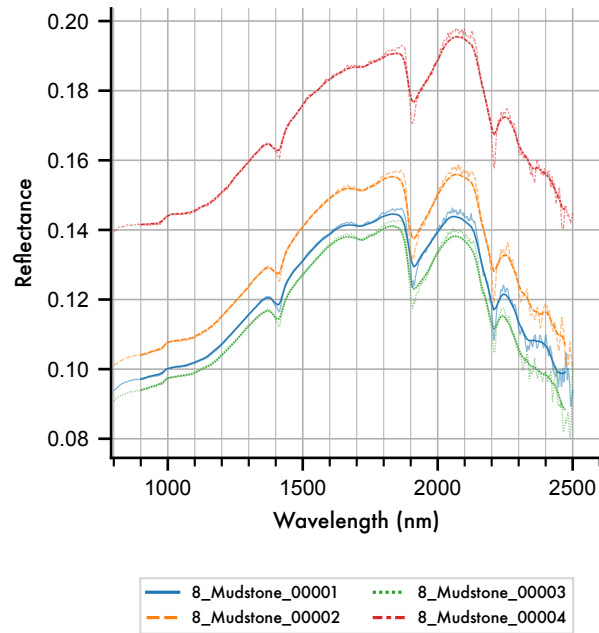


Figure 16 Mean of Noisy resampled example spectrum for Illite, showing mean and standard deviation of 64 repeat samples according to the spectral signal-to-noise ratio illustrated in figure 3.B.

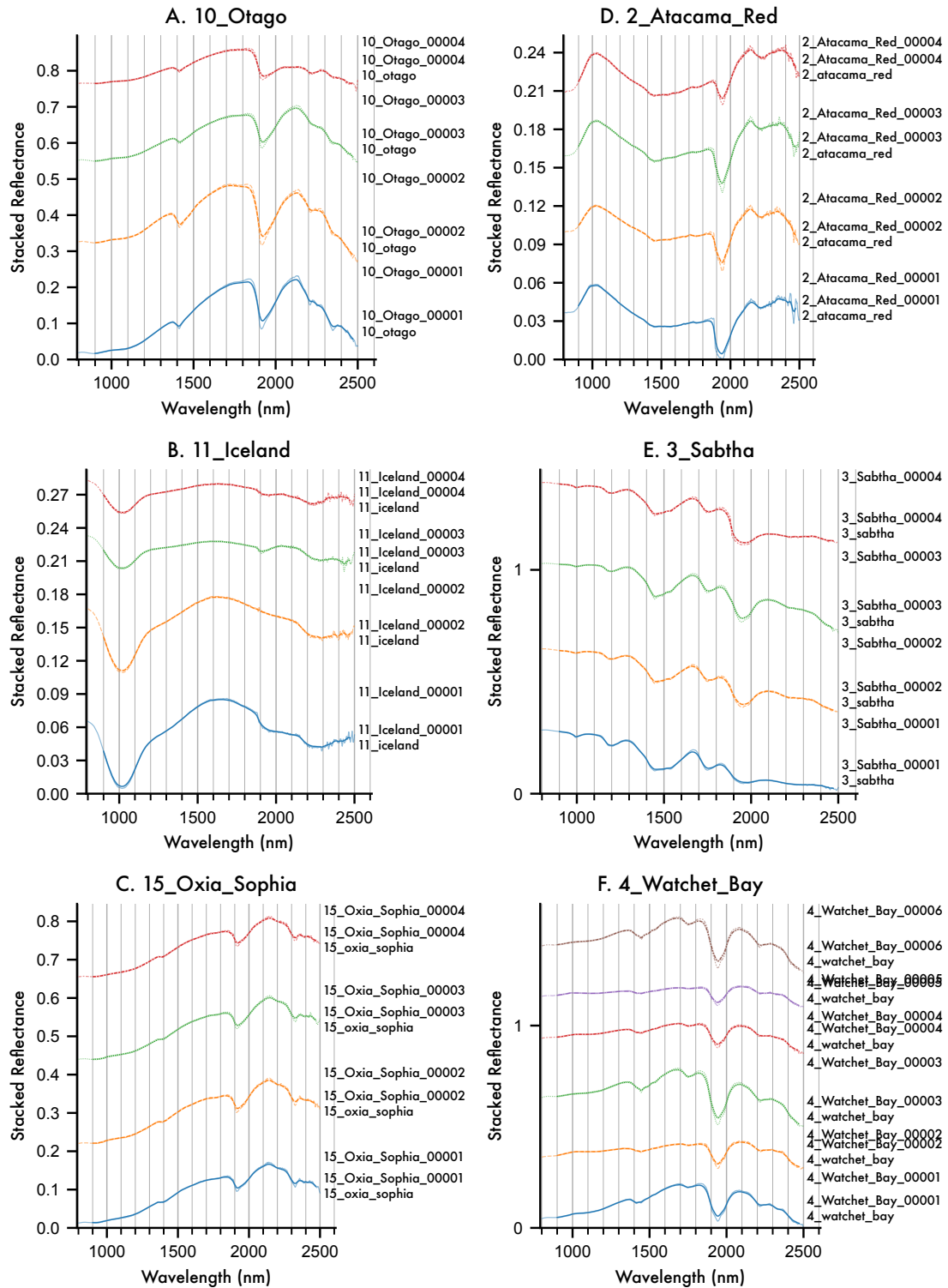
Here we find that the standard deviation is almost negligible at the scale of the figure.

Plotting out all of the groups, we again see that the noise, at this scale, is indeed negligible for most entries.

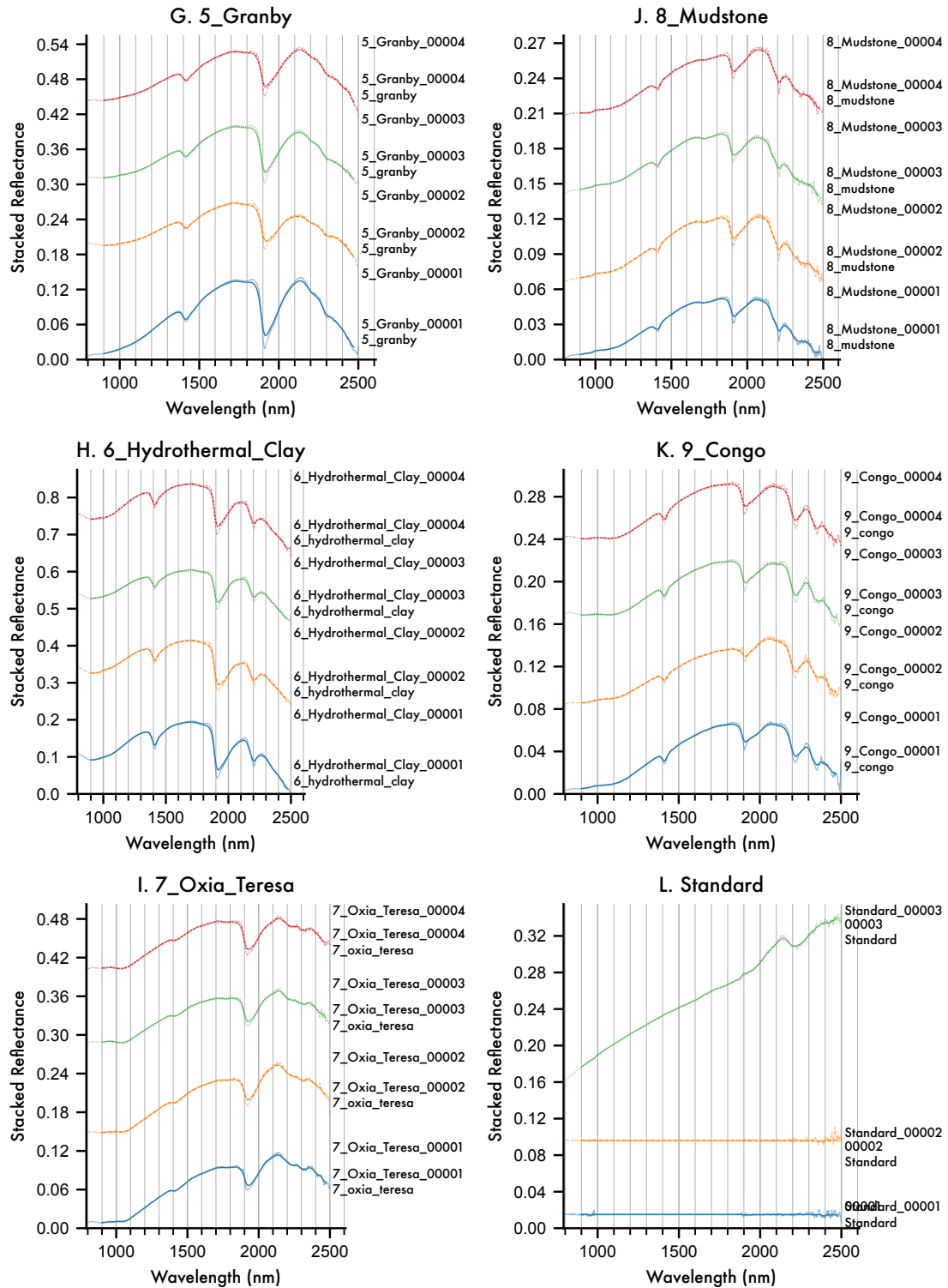
```
[267]: axes = obs.plot_profiles(scope='species', groupby='Sample ID', stacked=True,
    hires_under=True, ci=True)
```

Negative level values detected
 Negative level values detected
 Negative level values detected

Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id with Noise (1/3)



Enfys_Ingaas_Mwir Sampled Species grouped by Sample Id with Noise (2/3)



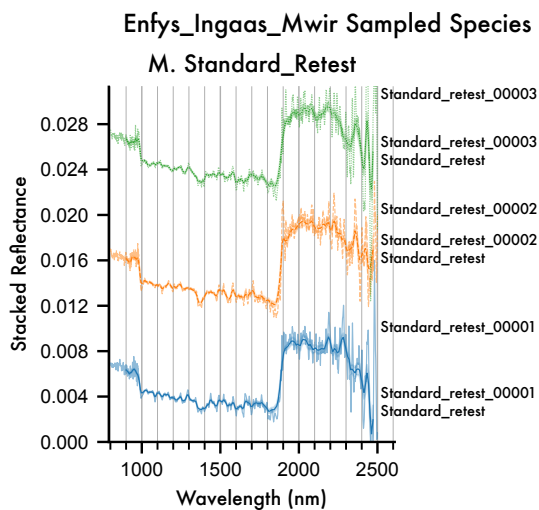


Figure 17 The MICA files as sampled by *Enfys*, with repeat samples drawn according to the expected spectral Signal-to-Noise Ratio, with the original high-resolution spectra plotted underneath each.

The key conclusion to draw from the above figure is that expected noise for *Enfys* does not significantly intrude on the diagnostic features of these type specimen of minerals identified on the Mars surface.

6 Conclusions

7 References